VANI TUITION CENTRE MANAGEMENT SYSTEM

Software Design Specification (SDS)

Document Version: 1.0
Date:12 June 2026

DOCUMENT REVISION HISTORY

Version	Date	Description	Author
1.0	12 June 2026	SDS draft document	Project Team

Table 0.1 – Document Revision History

1. INTRODUCTION

1.1 Purpose

This Software Design Specification (SDS) documents the complete architectural and detailed design of the Vani Tuition Centre Management System (VTCMS). The purpose of this document is to provide a comprehensive, authoritative reference for all design decisions that govern the implementation of the system. It translates the functional and non-functional requirements articulated in the Software Requirements Specification (SRS) into concrete design artefacts, including architectural diagrams, class structures, sequence interactions, state machines, database schemas, and user interface wireframes.

This document serves multiple audiences. Development engineers use it as a blueprint during coding. Quality assurance personnel use it as a baseline against which implemented behaviour is validated. Academic evaluators use it to assess the rigour, completeness, and appropriateness of the design decisions made by the project team. Accordingly, the document is written to the standard of IEEE Std 1016-2009, the internationally recognised standard for software design descriptions.

1.2 Scope

The Vani Tuition Centre Management System is a full-stack web application designed to digitise and streamline the administrative and academic operations of the Vani Tuition Centre. The system supports three primary user roles — Administrator, Teacher, and Student — and an external stakeholder role for Parents who receive automated notifications.

The system encompasses the following capability domains:

- Student registration, profile management, and record maintenance.
- Teacher registration, subject assignment, and salary payment tracking.
- QR code-based attendance recording with automated parent notification via SMS and email.
- Assignment creation, distribution, student submission, and teacher review with feedback.
- Marks entry, storage, and multi-stakeholder viewing across student, teacher, and parent roles.
- Notification management covering payment reminders, meeting notices, and general announcements.
- Attendance and academic reporting for administrative oversight.
- AI-powered learning support through a contextual chatbot integrated with the system's learning resource repository.

The system is delivered as a single-page application (SPA) frontend built with React.js, communicating with a Spring Boot REST API backend, persisting data in a MongoDB document database. The platform targets both desktop and mobile web browsers, requiring no installation on client devices beyond a modern browser.

1.3 References

- Vani Tuition Centre Management System – Software Requirements Specification (SRS), v1.0, 2026.
- IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions.
- Spring Boot Reference Documentation, v3.x, Pivotal Software, 2024.
- React Documentation, Meta Platforms Inc., 2024 (https://react.dev/).

- MongoDB Manual, v7.0, MongoDB Inc., 2024 (https://www.mongodb.com/docs/).
- OWASP Top Ten 2021 – Open Web Application Security Project (https://owasp.org/Top10/).
- JSON Web Token (JWT) – RFC 7519, Internet Engineering Task Force, 2015.

1.4 Overview

The remainder of this document is organised as follows. Section 2 provides a system overview, including a functional summary and the relationship of this design to the SRS. Section 3 presents the architectural design, covering the selected layered architecture, component diagram, deployment topology, and technology stack justification. Section 4 describes the detailed design through class diagrams, sequence diagrams, state machine diagrams, and database design. Section 5 addresses the user interface design with annotated wireframes and navigation flow analysis. Section 6 specifies external and internal interface designs. Section 7 analyses security design across authentication, data protection, input validation, and OWASP considerations. Section 8 maps every non-functional requirement from the SRS to a specific design decision with justification.

2. SYSTEM OVERVIEW

2.1 System Summary

The Vani Tuition Centre Management System (VTCMS) is a comprehensive, cloud-ready, multi-role web platform that replaces paper-based and fragmented digital workflows at a private tuition centre with an integrated, secure, and responsive management solution. The system is composed of three role-differentiated portals — an Admin Portal, a Teacher Portal, and a Student Portal — all served from a unified React.js frontend and supported by a common Spring Boot backend API.

The Context Diagram (Figure 2.1) illustrates the system's boundaries and data flows with its external actors and services. The four primary actors (Admin, Teacher, Student, and Parent) interact with the system through dedicated portals. Three external services integrate with the system: a QR Attendance Service that validates QR codes generated for class sessions, a Notification Gateway that routes SMS and email communications to parents, and an AI Learning Support Service that processes student queries against the learning resource repository.

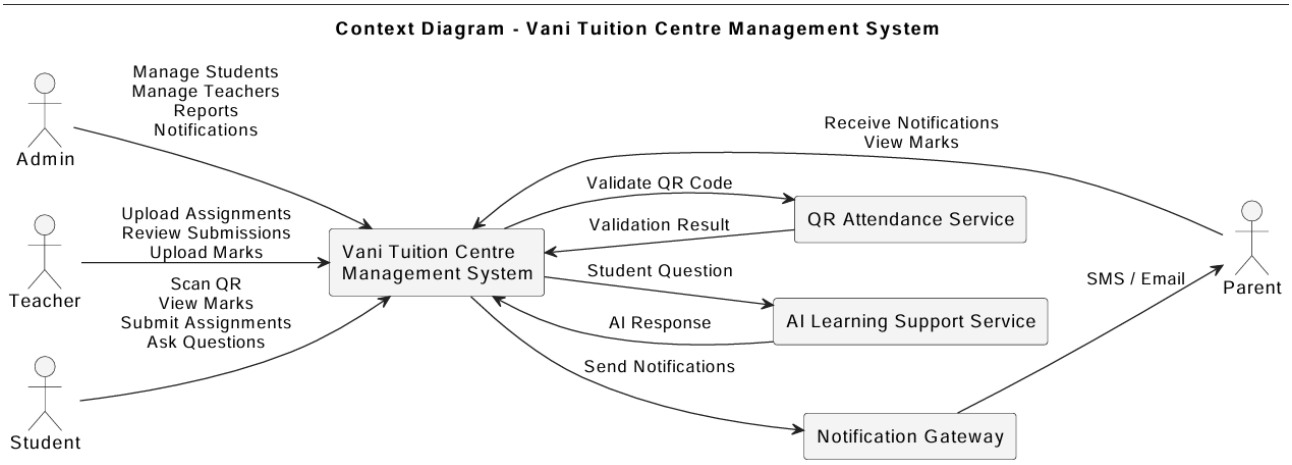

Figure 2.1 – Context Diagram – Vani Tuition Centre Management System

2.2 Relationship to SRS

This Software Design Specification is the direct design realisation of the requirements articulated in the Vani Tuition Centre Management System SRS. Every functional requirement defined in the SRS is addressed by one or more design artefacts within this document. The mapping is as follows:

SRS Requirement Area	Design Artefact(s)	Section(s)
Student Management	Class Diagram, SD-01 Sequence Diagram, Student Management UI	4.1, 4.2, 5.1
Teacher Management	Class Diagram, Teacher Management UI	4.1, 5.1
QR Attendance	SD-03 Sequence Diagram, SM-01 State Machine, ER Diagram	4.2, 4.3, 4.4
Marks Management	SD-07 Sequence Diagram, Marks UI	4.2, 5.1

SRS Requirement Area	Design Artefact(s)	Section(s)
Assignment Management	SD-06, SM-02, Assignment UI	4.2, 4.3, 5.1
Notification Management	SD-05, SM-03	4.2, 4.3
Attendance Reporting	SD-04, Attendance Report UI	4.2, 5.1
AI Learning Support	SD-08, SM-04, AI Chatbot UI	4.2, 4.3, 5.1
Database Design	ER Diagram, MongoDB Schema Design	4.4
Security Requirements	JWT Design, RBAC, OWASP Analysis	7

Table 2.1 – SRS to Design Artefact Mapping

2.3 Functional Overview

2.3.1 Student Management

The Student Management module provides the Administrator with comprehensive tools for maintaining the student population of the tuition centre. The Admin can add new students, update existing records, search by multiple criteria (ID, name, contact), and delete records following confirmation. The module also exposes attendance and payment views associated with each student record.

2.3.2 Teacher Management

Teacher Management enables the Administrator to register, update, delete, and view teacher records. This module also supports subject assignment — linking one or more subjects to a given teacher — and salary payment tracking. Teacher records include qualifications, joining dates, and employment status.

2.3.3 Attendance Management and QR Attendance

Attendance is captured through a QR code mechanism designed for mobile use. The system generates a unique, time-limited QR code for each class session. Students scan the QR code using their mobile device through the Student Portal. The system validates the code, records the attendance, and immediately triggers an automated parent notification via SMS/email.

2.3.4 Assignment Management

Teachers can create and publish assignments with file attachments and deadlines. Students view published assignments, submit their work, and track submission status. Teachers review submitted work and provide written feedback. The module tracks the full assignment lifecycle from creation through closure.

2.3.5 Marks Management

Teachers upload marks for specific assessments against individual students and subjects. Students can view their own marks; parents can view their child's marks. The system enforces role-based visibility at the data level.

2.3.6 Notification Management

The Admin can issue payment notifications, overdue payment reminders, parent meeting notices, and general announcements. The system retrieves recipient contact details, dispatches notifications via the integrated SMS/Email gateway, and records delivery status.

2.3.7 Reporting

On-demand filtered attendance reports are generated by the Admin specifying criteria such as student, class, and date range. The design anticipates future extension to payment and academic progress reports.

2.3.8 AI Learning Support

A chatbot interface within the Student Portal accepts natural language questions. The system analyses the query, searches the learning resource repository, and generates a contextually relevant AI-assisted response. Sessions are stateful, maintaining multi-turn dialogue history.

3. ARCHITECTURAL DESIGN

3.1 Architecture Pattern Selected

The Vani Tuition Centre Management System employs a Layered Architecture (N-Tier Architecture), organised into three principal tiers: the Presentation Layer, the Business Logic Layer, and the Data Layer. This pattern was selected over microservices, event-driven, or peer-to-peer alternatives because it aligns with the scale, team size, and delivery timeline of the project while producing a sound, maintainable, and extensible design.

3.1.1 Presentation Layer

Implemented as a React.js Single Page Application (SPA) executing in the user's browser. Responsible for all user interaction and communicates with the backend exclusively through RESTful HTTPS API calls. Subdivided into three portal contexts: Admin Portal, Teacher Portal, and Student Portal. Role-based routing enforced within the SPA ensures each authenticated user accesses only their appropriate portal. React components follow a container-component pattern: container components manage state and data fetching; stateless presentation components handle rendering.

3.1.2 Business Logic Layer

Implemented as a Spring Boot application exposing a RESTful API. Encapsulates all business rules, validation, authentication, and workflow orchestration. Internally organised into domain-specific service components: Student Service, Teacher Service, Attendance Service, Assignment Service, Marks Service, Notification Service, Payment Service, Report Service, and AI Learning Support Service. Cross-cutting concerns (JWT validation, input sanitisation, logging) are handled by Spring Boot middleware intercepting all requests before they reach service logic.

3.1.3 Data Layer

Implemented using MongoDB accessed through Spring Data MongoDB repository abstractions. Collections correspond to the entities identified in the ER Diagram. The layer also integrates with two external services: the SMS/Email notification gateway and the AI Processing Engine, both accessed exclusively from within the Business Logic Layer.

3.2 Architecture Justification

3.2.1 Maintainability

Strict separation of concerns means a schema change in the Data Layer does not propagate to the Business Logic Layer if the repository interface contract is preserved. UI changes do not affect business logic. This independence reduces cognitive load for maintenance and debugging.

3.2.2 Scalability

The stateless REST API (JWT-based, no server-side sessions) allows additional API server instances to be deployed behind a load balancer without session-sharing complexity. MongoDB supports horizontal scaling through sharding. The React SPA can be served from a CDN, offloading traffic from the application server.

3.2.3 Security

Security is enforced at every layer boundary. The Presentation Layer cannot directly access the database — all data access flows through the Business Logic Layer where JWT validation, RBAC checks, and input validation are applied consistently.

3.2.4 Reliability

Spring Boot provides production-grade health monitoring, exception handling, and transaction management. MongoDB's built-in replication supports automatic failover. The embedded Tomcat server reduces deployment complexity and configuration-related failure risk.

3.2.5 Extensibility

New functional modules can be added without restructuring existing layers. The REST API provides a stable integration contract allowing future clients (mobile native apps, third-party integrations) to consume system data without changes to the existing frontend.

3.3 Component Diagram

The Component Diagram (Figure 3.1) illustrates the high-level structural components of the system. The system is organised into three conceptual groups — Core Management Modules, Academic Modules, and Communication Modules — all integrated through the central Database component and external service interfaces.

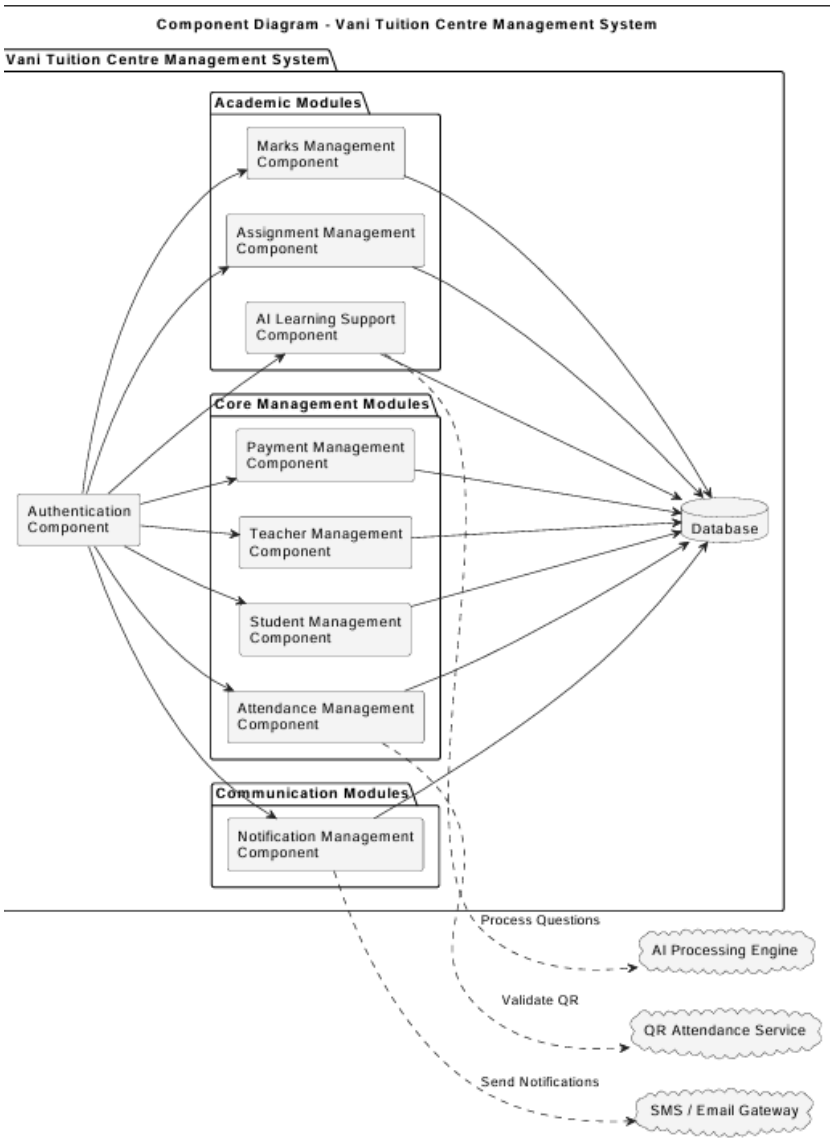

Figure 3.1 – Component Diagram – Vani Tuition Centre Management System

3.3.1 Component Descriptions

Authentication Component: Gateway of the entire system. Validates credentials, issues JWTs, and enforces token validation on every API request. Provides /auth/login and /auth/refresh endpoints, the only endpoints excluded from JWT validation.

Database Component: Central MongoDB data store. All other components interact through Spring Data MongoDB repository interfaces. No component exposes raw database query logic to others.

Student Management Component: Encapsulates all CRUD operations for student records. Provides sub-endpoints for attendance history and payment status within the student context.

Teacher Management Component: Mirrors Student Management in structure. Additionally provides subject assignment functionality linking teacher records to subject records through the teacher_subjects junction entity.

Attendance Management Component: Works in concert with the QR Attendance Service. Responsible for QR code generation for class sessions, storing session records, and receiving validated attendance data. Triggers Notification Service upon successful attendance recording.

Payment Management Component: Manages financial records for students (monthly fees) and teachers (salary). Provides the Admin with payment recording and status query interfaces.

Assignment Management Component: Handles the complete assignment lifecycle. Exposes teacher-side operations (create, publish, review) and student-side operations (view, submit).

Marks Management Component: Provides endpoints for teachers to upload marks and for students and parents to retrieve marks with role-based visibility enforcement.

AI Learning Support Component: Integrates with the AI Processing Engine external service. Forwards student questions along with relevant learning resource context and returns generated responses.

Notification Management Component: Interfaces with the SMS/Email Gateway. Manages notification records and updates delivery status based on gateway responses.

3.4 Deployment Diagram

The Deployment Diagram (Figure 3.2) illustrates the physical and logical distribution of system components across computing nodes, designed for deployment on a cloud hosting platform.

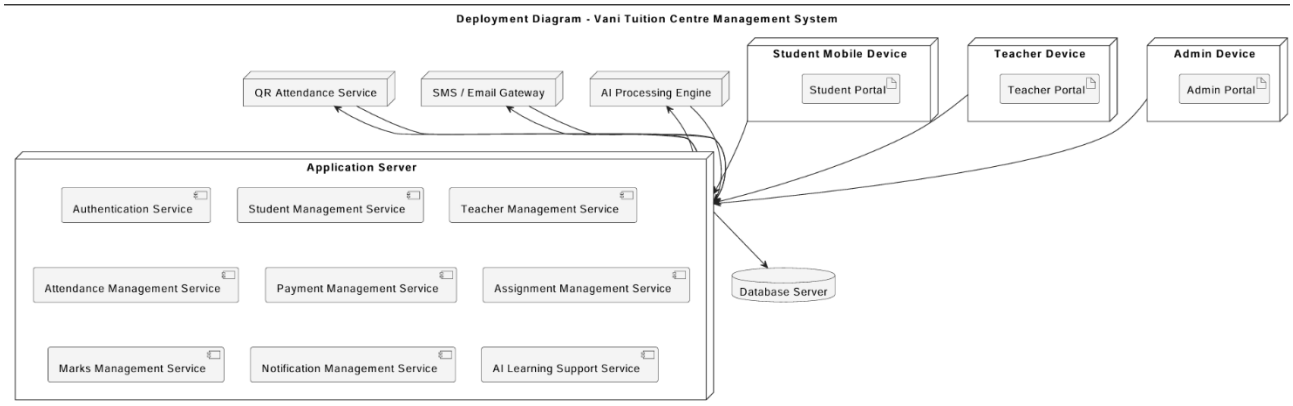

Figure 3.2 – Deployment Diagram – Vani Tuition Centre Management System

3.4.1 Client Devices

Three categories of client device interact with the system: Student Mobile Devices (smartphones and tablets used for QR attendance scanning), Teacher Devices (desktop or laptop computers for classroom management), and Admin Devices (desktop computers for administrative operations). All access the system exclusively through a standard web browser with no native installation required.

3.4.2 React Frontend

The React SPA is served as static files (HTML, JavaScript, CSS) from the Application Server or, in a production-optimised deployment, from a CDN edge node. The SPA communicates with the Spring Boot API through HTTPS REST calls and enforces role-based routing based on the authenticated user's JWT role claim.

3.4.3 Spring Boot Application Server

Hosts the Spring Boot application. Processes all API requests, applies JWT validation, invokes appropriate service components, and returns JSON responses. Configured with an embedded Tomcat server, eliminating the need for separate web server configuration.

3.4.4 MongoDB Database Server

Hosts the MongoDB instance, accessible only from the Application Server. In production, configured with replica sets for high availability and automatic failover. Nightly backups are written to a separate storage location.

3.4.5 External Services

The QR Attendance Service handles QR code validation logic via an internal API. The SMS/Email Gateway (e.g., Twilio or SendGrid) delivers notifications to parent phone numbers and email addresses. The AI Processing Engine processes natural language queries and generates contextual educational responses.

3.5 Technology Stack Justification

3.5.1 React.js

Selected for its component-based architecture promoting reusable, testable UI components. Its virtual DOM delivers high rendering performance. React Router provides client-side routing; Axios handles HTTP communication. React's widespread adoption ensures extensive documentation and community support are available.

3.5.2 Spring Boot

Provides a production-grade, convention-over-configuration framework for REST APIs in Java. Spring Security delivers mature JWT authentication and RBAC. Spring Data MongoDB provides the repository abstraction decoupling business logic from database access. Strong Java typing and compile-time error detection are particularly valuable for implementing business rules with high correctness assurance.

3.5.3 MongoDB

The document model aligns naturally with the JSON data exchanged between React and Spring Boot, minimising object-relational impedance mismatch. Flexible schema supports rapid iteration. Native support for nested documents is advantageous for entities such as notifications with recipient lists. MongoDB Atlas provides a managed cloud database service.

3.5.4 JWT Authentication

Enables stateless API security. JWTs encode identity and role directly within the signed token, allowing the API server to validate any request without consulting a session store. Tokens are transmitted via the HTTP Authorization Bearer scheme. Token expiry enforces periodic re-authentication.

3.5.5 REST APIs

Selected for simplicity, statelessness, and alignment with standard HTTP semantics. Standard HTTP methods (GET, POST, PUT, DELETE) and status codes make the API discoverable and testable with tools such as Postman. Any future client can consume the API without client-specific protocol libraries.

4. DETAILED DESIGN

4.1 Class Diagram

The Class Diagram (Figure 4.1) describes the object-oriented structure of the system. It defines all classes, their attributes and methods, and the associations between them. Each class corresponds to a Java entity annotated with Spring Data MongoDB mappings, and maps directly to a MongoDB collection in the Data Layer.

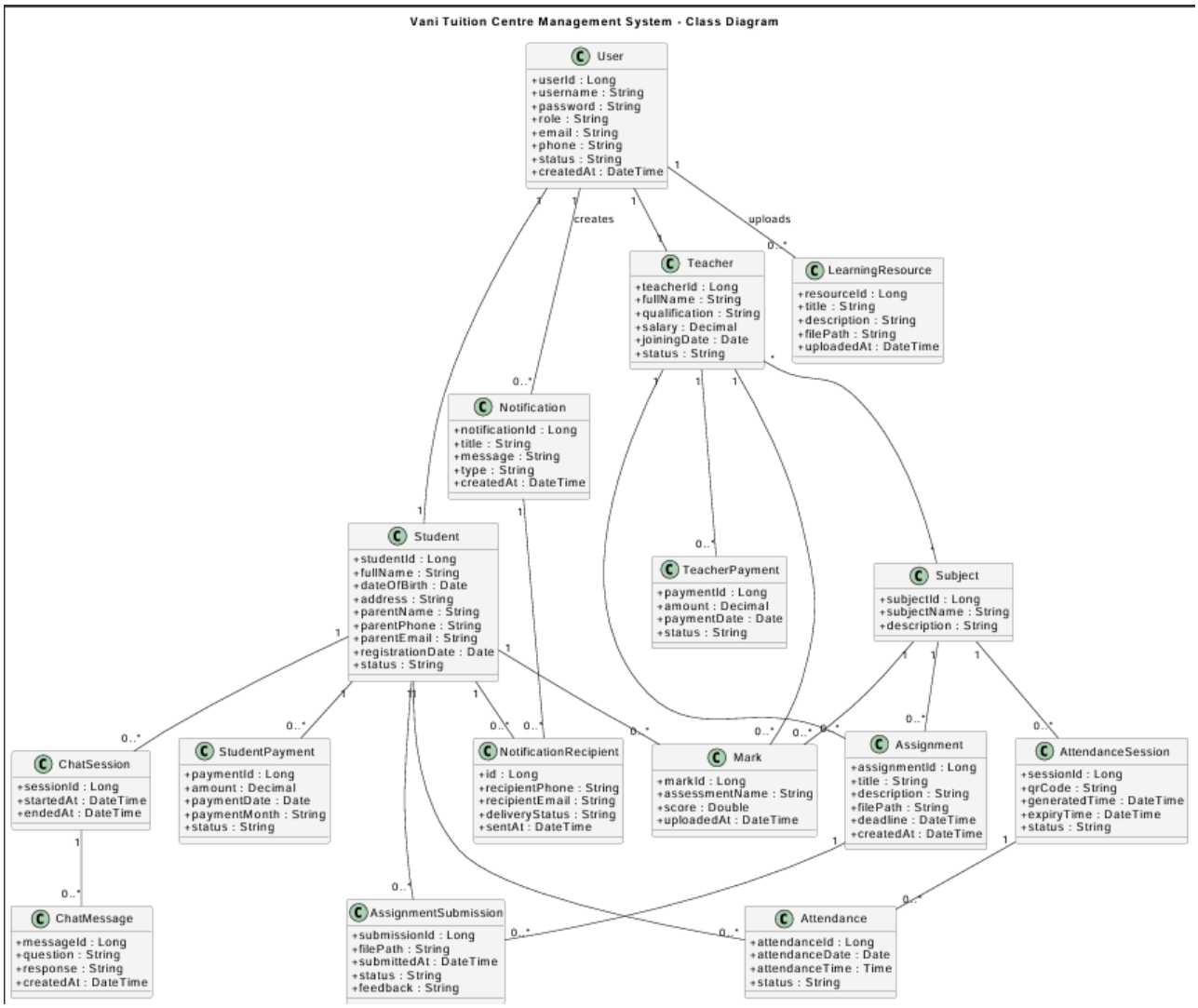

Figure 4.1 – Class Diagram – Vani Tuition Centre Management System

4.1.1 Class Descriptions

User: Root of the authentication hierarchy. Fields: userId, username, password (BCrypt hash), role (ADMIN/TEACHER/STUDENT), email, phone, status, createdAt. One-to-one association with Student and Teacher.

Student: Enrolled student personal and parental data. Fields: studentId, userId (FK), fullName, dateOfBirth, address, parentName, parentPhone, parentEmail, registrationDate, status. One-to-many with Attendance, AssignmentSubmission, Mark, StudentPayment.

Teacher: Instructor professional information. Fields: teacherId, userId (FK), fullName, qualification, salary, joiningDate, status. Many-to-many with Subject (via teacher_subjects). One-to-many with Assignment, Mark, TeacherPayment.

Subject: Academic subject catalogue. Fields: subjectId, subjectName, description. Many-to-many with Teacher; one-to-many with AttendanceSession, Assignment, Mark.

AttendanceSession: A class session with a QR code. Fields: sessionId, subjectId (FK), qrCode, generatedTime, expiryTime, status (ACTIVE/EXPIRED). One-to-many with Attendance.

Attendance: Individual student attendance record. Fields: attendanceId, studentId (FK), sessionId (FK), attendanceDate, attendanceTime, status (PRESENT/ABSENT/LATE).

Assignment: Task published by a teacher. Fields: assignmentId, teacherId, subjectId, title, description, filePath, deadline, createdAt. One-to-many with AssignmentSubmission.

AssignmentSubmission: Student response to an assignment. Fields: submissionId, assignmentId (FK), studentId (FK), filePath, submittedAt, status (SUBMITTED/REVIEWED/LATE), feedback.

Mark: Assessment score record. Fields: markId, studentId, subjectId, teacherId (all FK), assessmentName, score, uploadedAt. Triple FK enables querying from any perspective.

Notification & NotificationRecipient: Notification: notificationId, title, message, type, createdBy, createdAt. NotificationRecipient: per-recipient delivery tracking with phone, email, deliveryStatus, sentAt.

LearningResource: Educational materials for AI-assisted learning. Fields: resourceId, title, description, filePath, uploadedBy, uploadedAt.

ChatSession & ChatMessage: ChatSession: session record with startedAt, endedAt, linked to one Student. ChatMessage: individual question-response pair with question, response, createdAt.

4.2 Sequence Diagrams

The following sequence diagrams illustrate the runtime interactions between actors, user interface components, service objects, and data stores for each key use case. Each diagram is accompanied by a detailed narrative explaining actors, message flow, and alternative scenarios.

4.2.1 SD-01: Manage Student Records

Illustrates the Admin workflow for any student record operation (add, update, delete, search, view attendance, view payment).

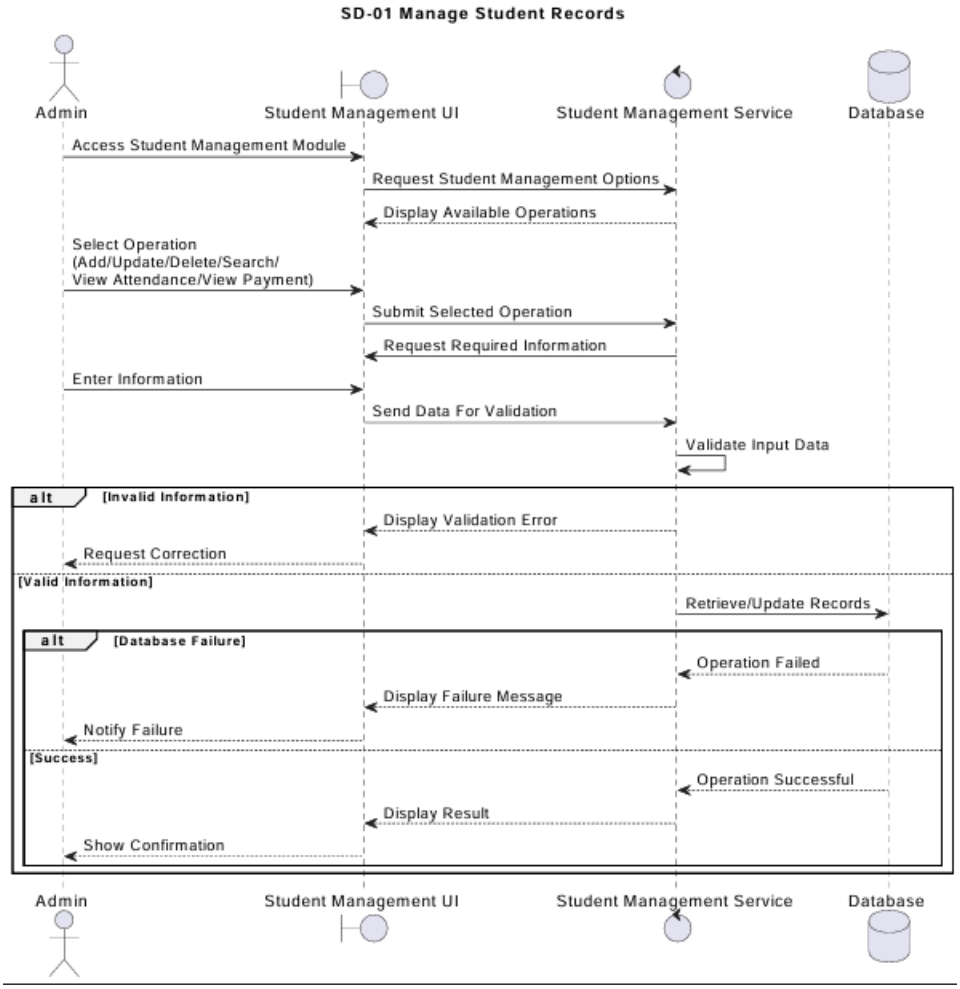

Figure 4.2 – SD-01 Sequence Diagram – Manage Student Records

Actors: Admin, Student Management UI, Student Management Service, Database.

Workflow: Admin accesses the Student Management module. UI displays available operations. Admin selects an operation and provides required input. UI submits data to the Student Management Service, which validates input. On valid input, the service retrieves or updates the Database and returns a result to the UI, which displays a confirmation.

Alternative Flows: (1) Invalid input triggers a validation error; the UI displays the error and prompts correction. (2) Database failure triggers an exception caught by the service; the UI displays a failure notification.

4.2.2 SD-03: QR Attendance Process

The most complex multi-actor flow in the system, involving the Student, the Student Web Application, the QR Attendance Service, the Database, and the Notification Service.

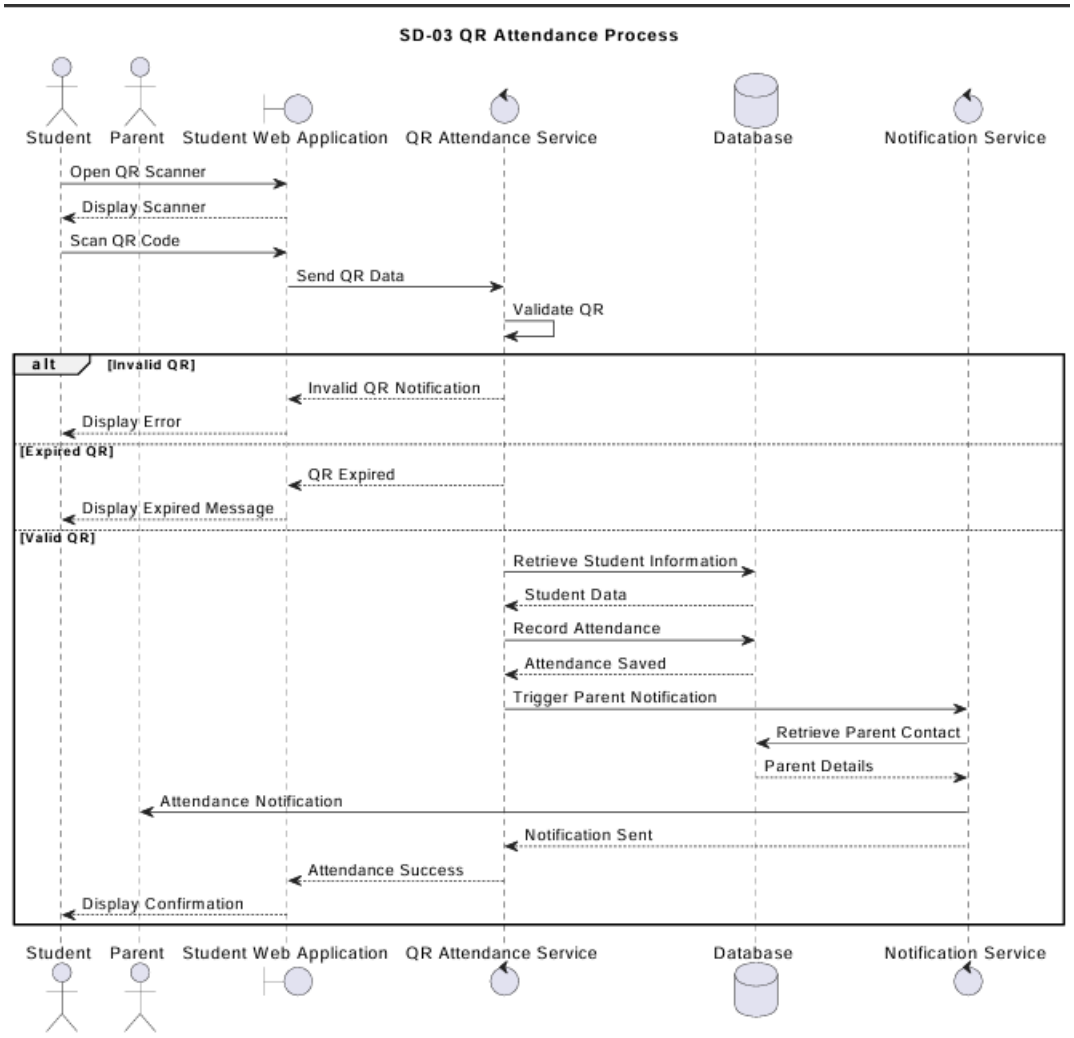

Figure 4.3 – SD-03 Sequence Diagram – QR Attendance Process

Actors: Student, Parent (recipient), Student Web Application, QR Attendance Service, Database, Notification Service.

Workflow: Student opens the QR scanner and scans the session code. Application sends the QR data to the QR Attendance Service for validation. On a valid, non-expired code: the service retrieves student information, records the attendance entry in the Database, triggers the Notification Service, which retrieves parent contact details and dispatches an attendance notification. The UI displays a confirmation to the Student.

Alternative Flows: (1) Invalid QR — service returns an invalid QR error; UI displays an error notification. (2) Expired QR — service returns an expiry message; UI prompts the student to request a new QR code from the teacher.

4.2.3 SD-04: Generate Attendance Report

Describes the Admin's workflow for generating a filtered attendance report.

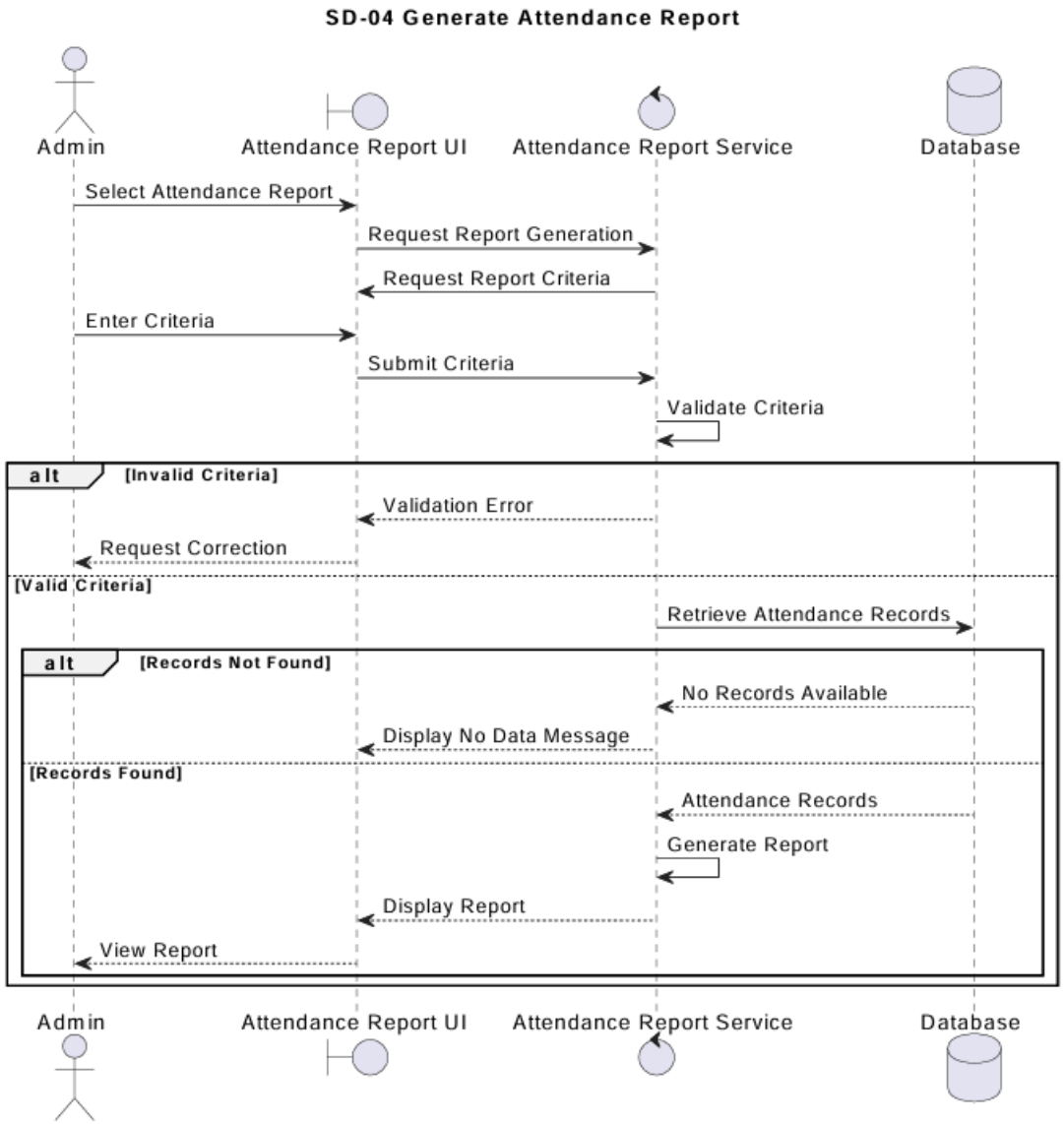

Figure 4.4 – SD-04 Sequence Diagram – Generate Attendance Report

Actors: Admin, Attendance Report UI, Attendance Report Service, Database.

Workflow: Admin selects report option and enters criteria (student, date range, class). UI submits criteria to the Attendance Report Service, which validates criteria and queries the Database. If records are found, the service generates the report returned to the UI. If no records match, the UI displays a no-data message.

4.2.4 SD-05: Manage Notifications

Covers the Admin's notification management workflow including creation, dispatch, and delivery tracking.

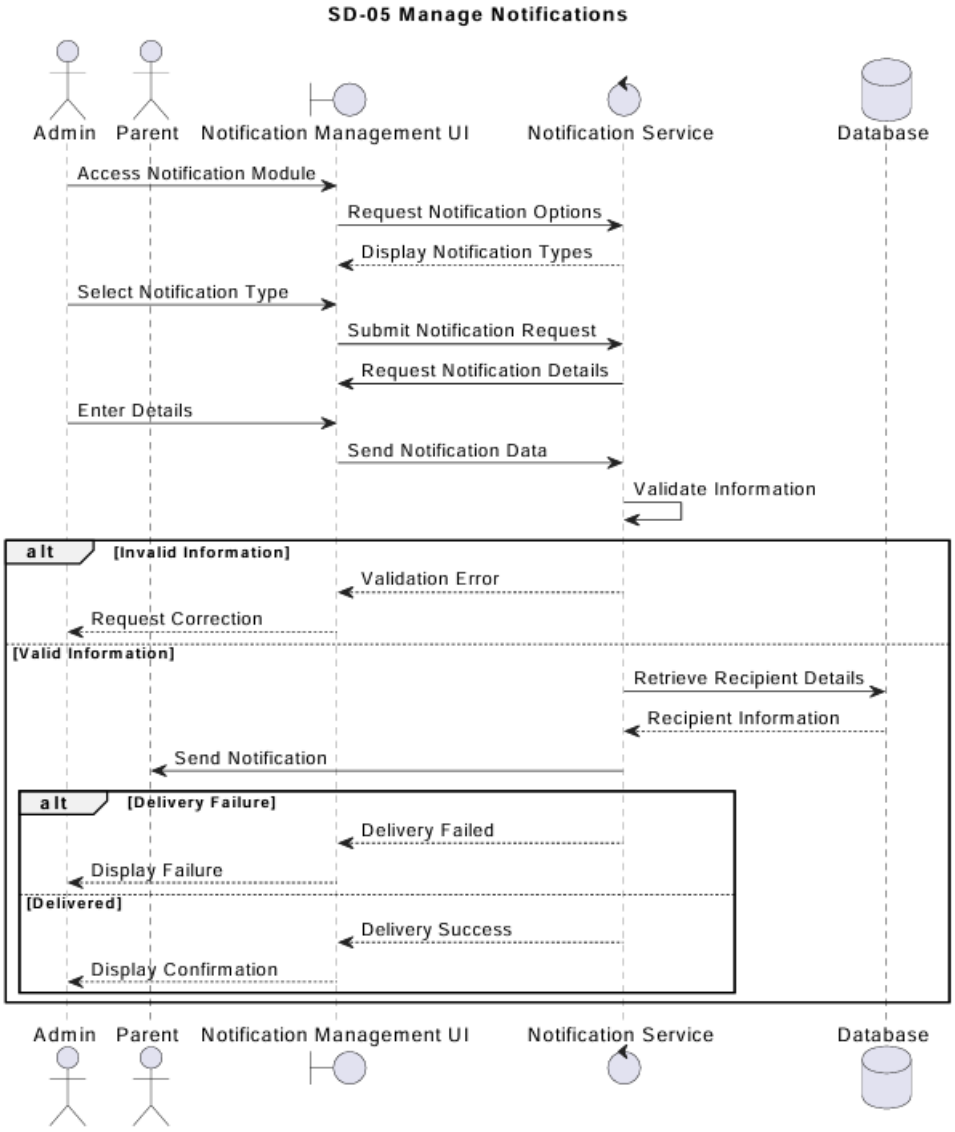

Figure 4.5 – SD-05 Sequence Diagram – Manage Notifications

Actors: Admin, Parent (recipient), Notification Management UI, Notification Service, Database.

Workflow: Admin accesses the Notification module and selects a notification type. On submission, the Notification Service validates the information, retrieves recipient details from the Database, and dispatches the notification via the external SMS/Email gateway. Delivery status is recorded, and the UI displays a confirmation.

Alternative Flows: (1) Validation failure returns an error to the UI. (2) Gateway delivery failure is recorded in the Database and reported to the Admin.

4.2.5 SD-06: Assignment Management

Covers the complete assignment lifecycle from teacher upload through student submission and teacher review.

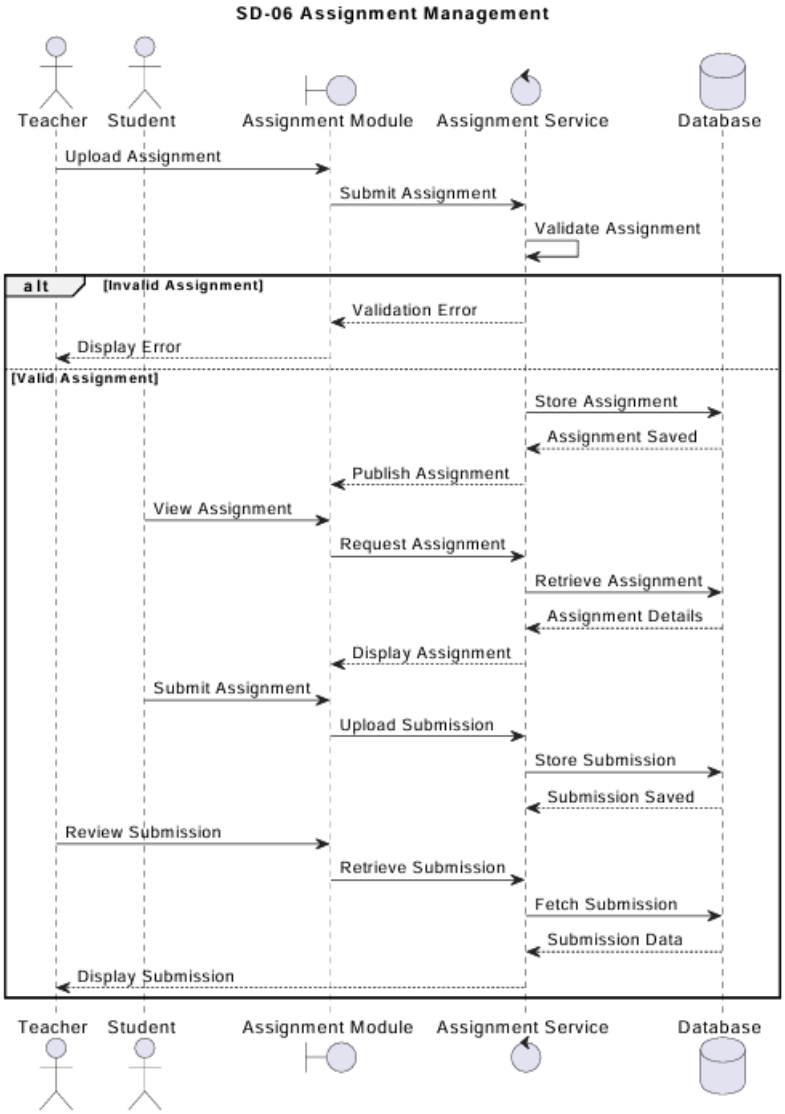

Figure 4.6 – SD-06 Sequence Diagram – Assignment Management

Actors: Teacher, Student, Assignment Module (UI), Assignment Service, Database.

Teacher workflow: Teacher uploads an assignment; Assignment Service validates, stores, and publishes it. Student workflow: Student views the published assignment, downloads details, and uploads a submission. Teacher review: Teacher retrieves and reviews the submission, adding feedback which the service stores.

4.2.6 SD-07: Marks Management

Covers the marks upload workflow by teachers and the viewing workflow by students and parents.

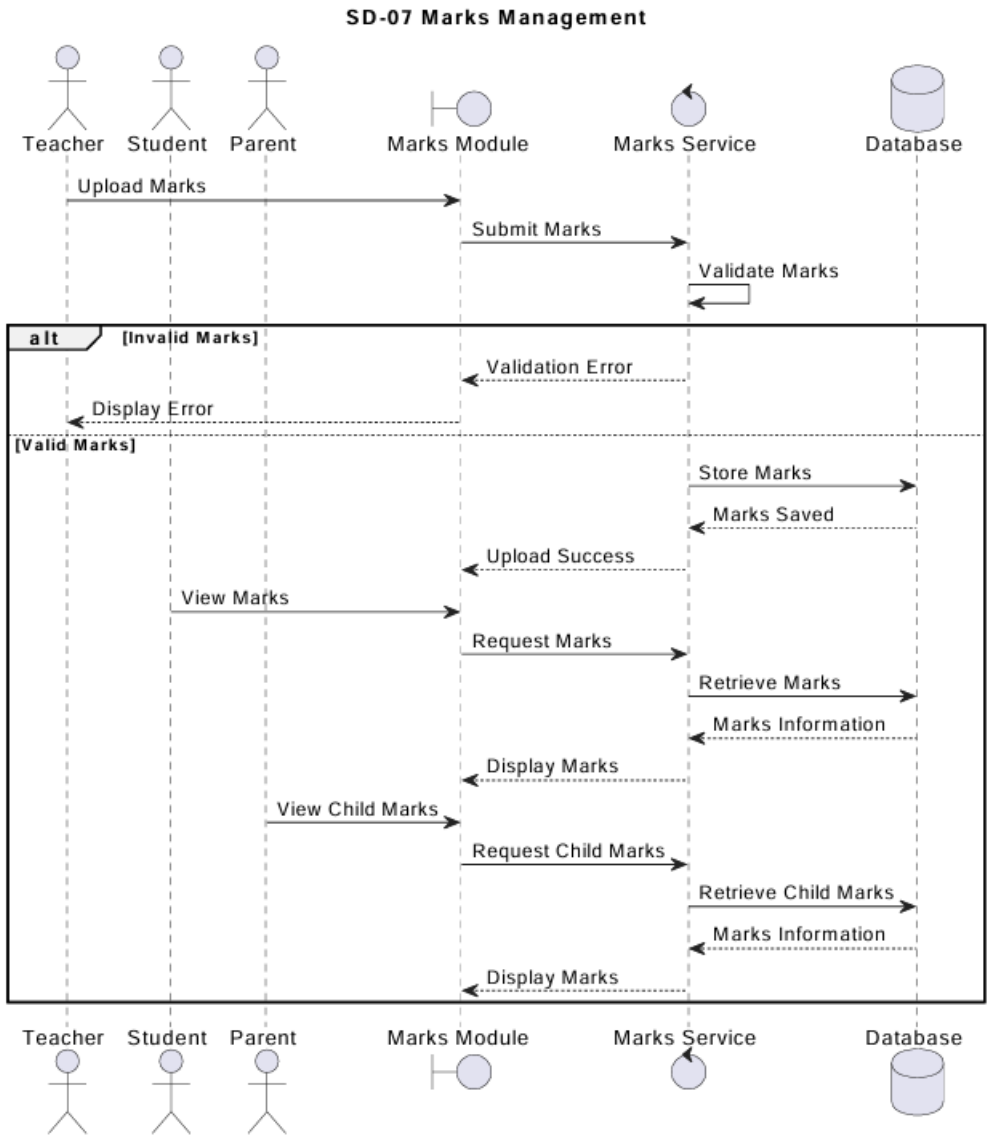

Figure 4.7 – SD-07 Sequence Diagram – Marks Management

Actors: Teacher, Student, Parent, Marks Module (UI), Marks Service, Database.

Workflow: Teacher uploads marks through the Marks Module. Marks Service validates and stores them in the Database. Student requests their own marks; Marks Service enforces ownership and returns only the requesting student's records. Parent requests child marks; Marks Service validates the parent-child relationship before returning results.

4.2.7 SD-08: AI Learning Support

Describes the AI chatbot interaction flow from session initialisation through multi-turn dialogue.

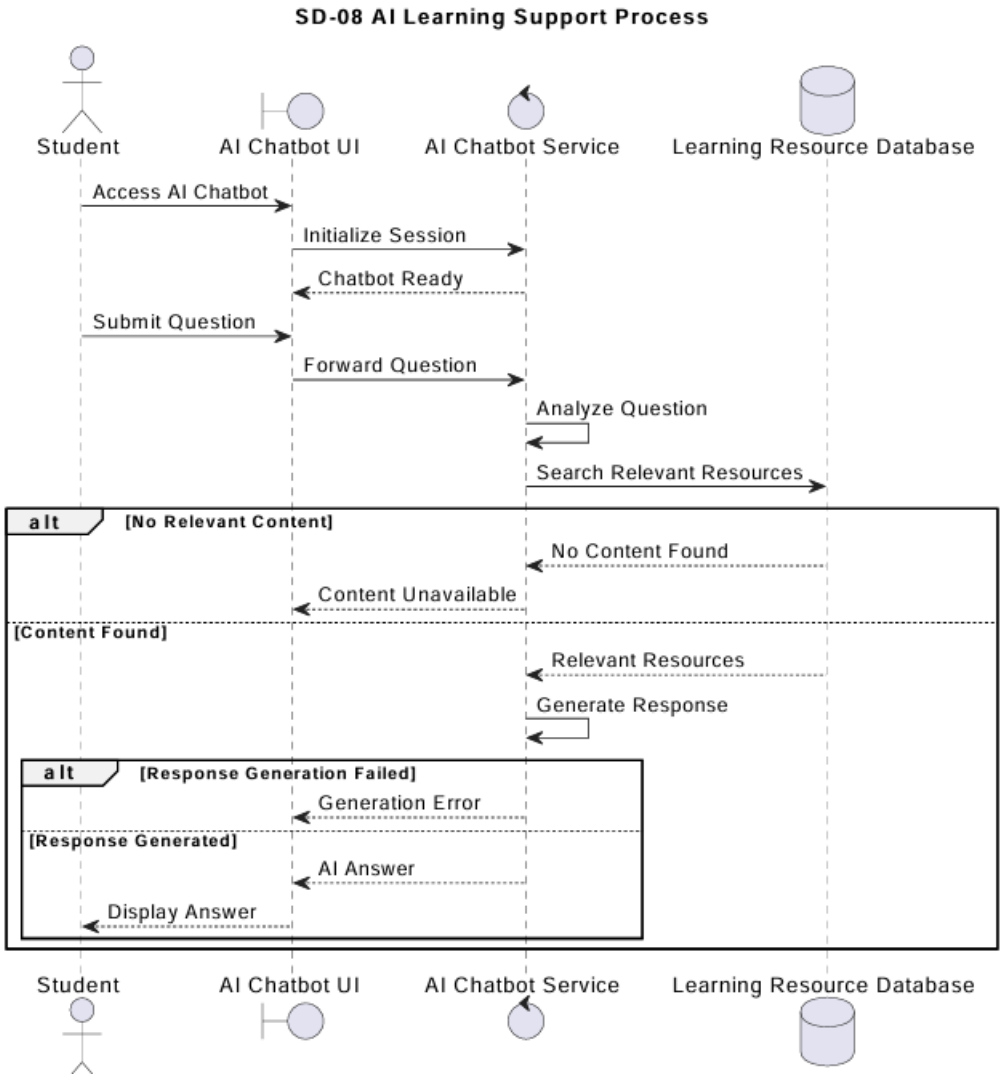

Figure 4.8 – SD-08 Sequence Diagram – AI Learning Support Process

Actors: Student, AI Chatbot UI, AI Chatbot Service, Learning Resource Database.

Workflow: Student opens the AI Chatbot; a session record is created. Student submits a question which is forwarded to the AI Chatbot Service. The service analyses the question, searches the Learning Resource Database for relevant content, and passes matching resources to the AI Processing Engine to generate a response. The response is displayed to the Student. Failure states (no content found, generation error) display appropriate fallback messages.

4.3 State Machine Diagrams

4.3.1 SM-01: Attendance State Machine

Models the lifecycle of a student's attendance event from QR scan initiation to completion of parent notification.

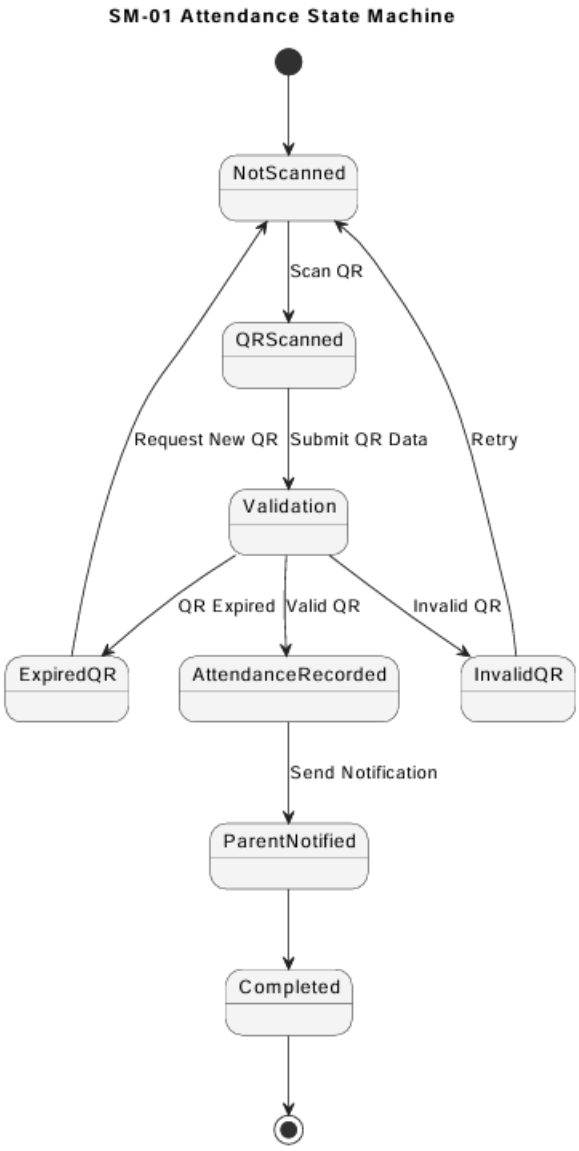

Figure 4.9 – SM-01 Attendance State Machine

States: NotScanned (initial) → QRScanned → Validation → [ExpiredQR | InvalidQR | AttendanceRecorded] → ParentNotified → Completed (final).

Key transitions: Scan QR moves from NotScanned to QRScanned. Submit QR Data moves to Validation. From Validation: Valid QR → AttendanceRecorded; QR Expired → ExpiredQR (can retry by requesting new QR); Invalid QR → InvalidQR (can retry). Send Notification moves from AttendanceRecorded → ParentNotified → Completed.

4.3.2 SM-02: Assignment State Machine

Models the lifecycle of a single assignment from teacher creation through final closure.

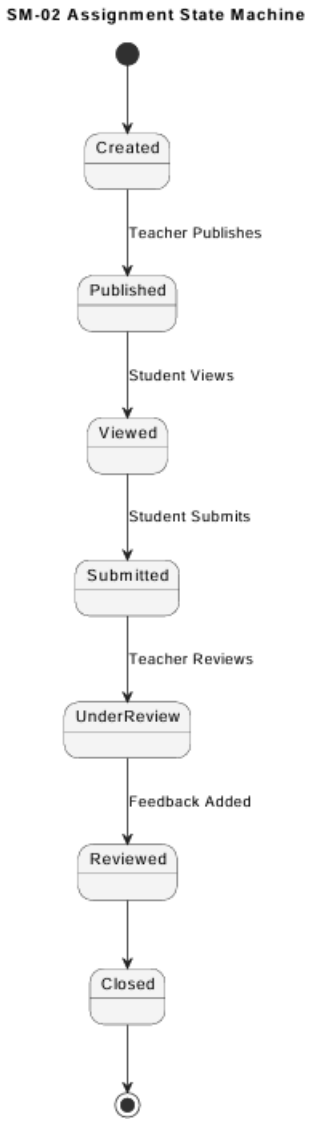

Figure 4.10 – SM-02 Assignment State Machine

States: Created → Published → Viewed → Submitted → UnderReview → Reviewed → Closed.

Transitions: Teacher Publishes (Created → Published). Student Views (Published → Viewed). Student Submits (Viewed → Submitted). Teacher Reviews (Submitted → UnderReview). Feedback Added (UnderReview → Reviewed). Assignment deadline or explicit close action moves to Closed.

4.3.3 SM-03: Notification State Machine

Captures the lifecycle of a notification from Admin creation to archival.

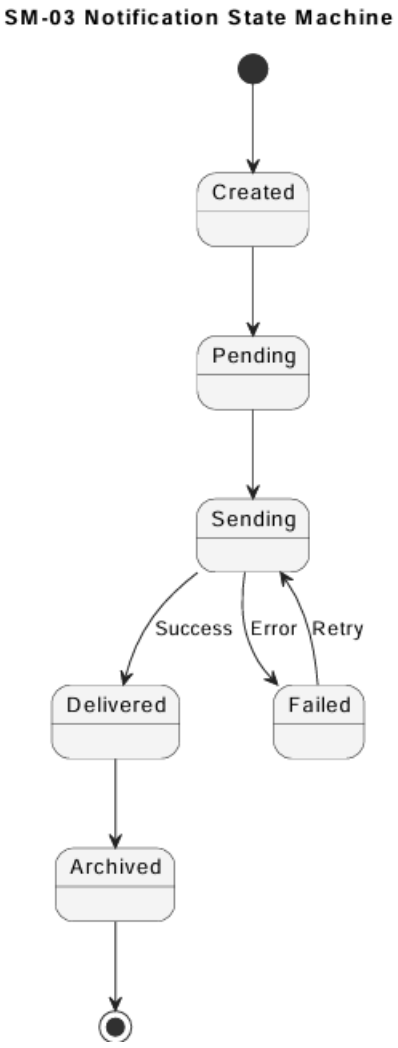

Figure 4.11 – SM-03 Notification State Machine

States: Created → Pending → Sending → [Delivered | Failed] → Archived.

Transitions: Admin submission moves Created → Pending. Notification Service dispatch moves Pending → Sending. Gateway Success → Delivered; Gateway Error → Failed (retry moves back to Sending). Both Delivered and Failed are eventually archived.

4.3.4 SM-04: AI Chat Session State Machine

Models the lifecycle of a single AI learning support session.

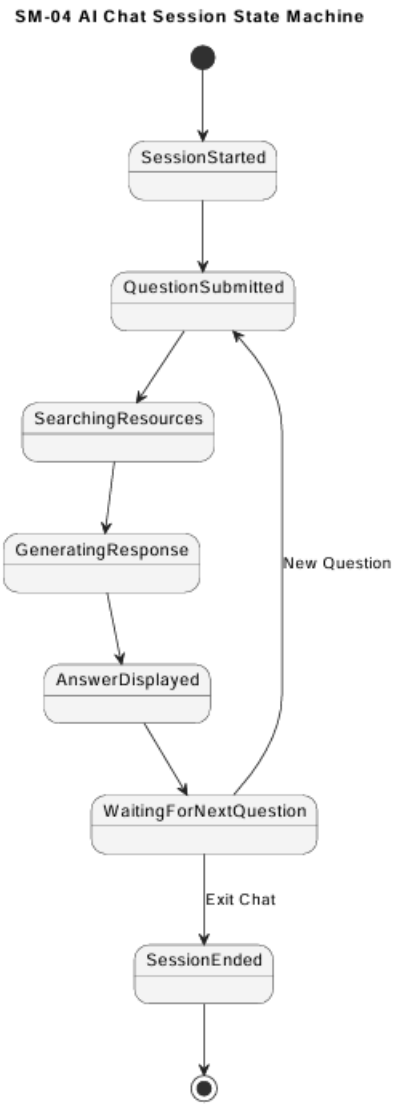

Figure 4.12 – SM-04 AI Chat Session State Machine

States: SessionStarted → QuestionSubmitted → SearchingResources → GeneratingResponse → AnswerDisplayed → WaitingForNextQuestion → SessionEnded.

Transitions: Student's first question moves SessionStarted → QuestionSubmitted. Service processing transitions through SearchingResources → GeneratingResponse → AnswerDisplayed. New Question from WaitingForNextQuestion loops back to QuestionSubmitted enabling multi-turn dialogue. Exit Chat moves to SessionEnded.

4.4 Database Design

The ER Diagram (Figure 4.13) presents the conceptual data model of the system. The MongoDB implementation uses BSON document collections that reflect this relational structure, with application-level foreign key references enforced by the Business Logic Layer.

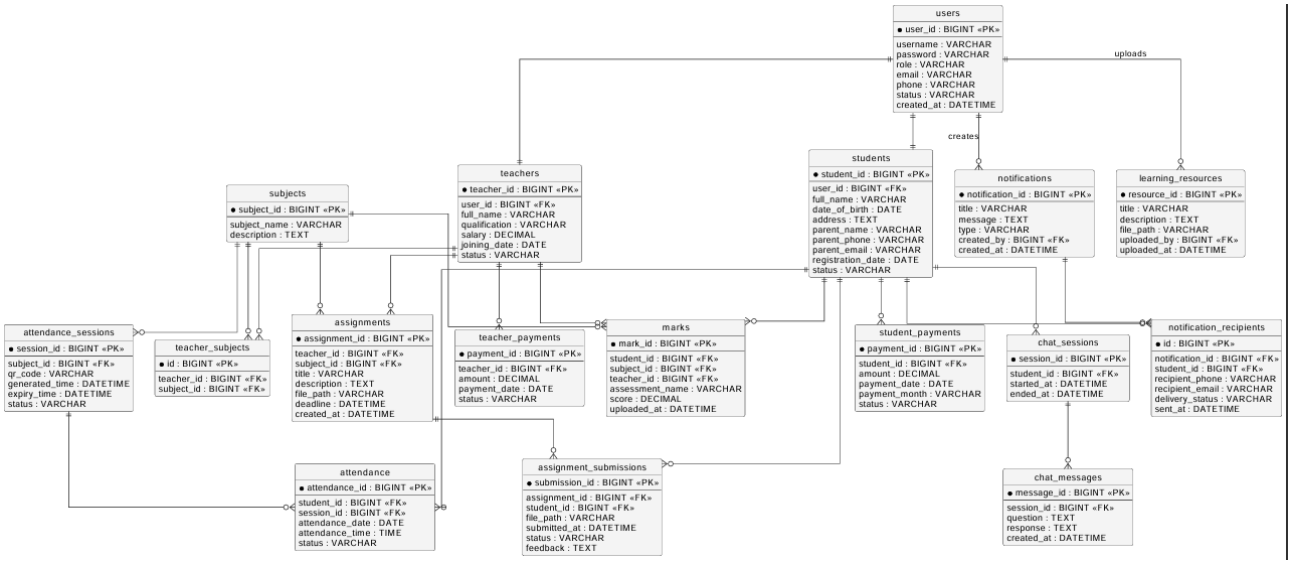

Figure 4.13 – ER Diagram – Vani Tuition Centre Management System

4.4.1 Entity Descriptions

users: Central authentication entity. Fields: user_id (PK), username (unique), password (BCrypt hash), role (ADMIN/TEACHER/STUDENT), email, phone, status, created_at. One-to-one with students and teachers.

students: Student personal and parental contact data. Fields: student_id (PK), user_id (FK), full_name, date_of_birth, address, parent_name, parent_phone, parent_email, registration_date, status. One-to-many with attendance, student_payments, assignment_submissions, marks.

teachers: Teacher professional data. Fields: teacher_id (PK), user_id (FK), full_name, qualification, salary (DECIMAL), joining_date, status. Many-to-many with subjects; one-to-many with assignments, marks, teacher_payments.

subjects: Academic subject catalogue. Fields: subject_id (PK), subject_name, description. Many-to-many with teachers (via teacher_subjects); one-to-many with attendance_sessions, assignments, marks.

teacher_subjects: Junction entity resolving the teacher-subject many-to-many relationship. Fields: id (PK), teacher_id (FK), subject_id (FK).

attendance_sessions: Class session with QR code. Fields: session_id (PK), subject_id (FK), qr_code (unique), generated_time, expiry_time, status (ACTIVE/EXPIRED). One-to-many with attendance.

attendance: Individual student attendance record. Fields: attendance_id (PK), student_id (FK), session_id (FK), attendance_date, attendance_time, status (PRESENT/ABSENT/LATE).

student_payments: Monthly fee payment records. Fields: payment_id (PK), student_id (FK), amount (DECIMAL), payment_date, payment_month, status (PAID/PENDING/OVERDUE).

teacher_payments: Salary payment records. Fields: payment_id (PK), teacher_id (FK), amount (DECIMAL), payment_date, status (PAID/PENDING).

assignments: Teacher-created tasks. Fields: assignment_id (PK), teacher_id (FK), subject_id (FK), title, description, file_path, deadline, created_at. One-to-many with assignment_submissions.

assignment_submissions: Student submission records. Fields: submission_id (PK), assignment_id (FK), student_id (FK), file_path, submitted_at, status (SUBMITTED/REVIEWED/LATE), feedback (TEXT).

marks: Assessment scores. Fields: mark_id (PK), student_id (FK), subject_id (FK), teacher_id (FK), assessment_name, score (DECIMAL), uploaded_at. Triple FK enables querying from student, subject, or teacher perspective.

notifications: Admin-created communications. Fields: notification_id (PK), title, message, type (PAYMENT/REMINDER/MEETING/ANNOUNCEMENT), created_by (FK→users), created_at. One-to-many with notification_recipients.

notification_recipients: Per-recipient delivery tracking. Fields: id (PK), notification_id (FK), student_id (FK), recipient_phone, recipient_email, delivery_status (DELIVERED/FAILED/PENDING), sent_at.

learning_resources: Educational materials for AI learning. Fields: resource_id (PK), title, description, file_path, uploaded_by (FK→users), uploaded_at.

chat_sessions: AI chat session records. Fields: session_id (PK), student_id (FK), started_at, ended_at. One-to-many with chat_messages.

chat_messages: Individual question-response pairs within a session. Fields: message_id (PK), session_id (FK), question (TEXT), response (TEXT), created_at.

4.4.2 Key Relationships and Constraints

The users table is the root identity model. Every student and teacher has a corresponding user record linked by user_id. The teacher_subjects junction table resolves the many-to-many teacher-subject relationship. The attendance model chains through attendance_sessions to subjects, correlating attendance with specific classes. The notification model uses a separate notification_recipients table for bulk delivery with independent per-recipient delivery tracking. All foreign key relationships are enforced at the application level by the Business Logic Layer service methods.

5. USER INTERFACE DESIGN

5.1 User Interface Design

The user interface of the Vani Tuition Centre Management System follows a consistent, role-differentiated design language. All screens share a common framework — a fixed left navigation sidebar, a top header bar showing the authenticated user's name and logout button, and a main content area. This consistency reduces the learning curve and ensures predictable navigation conventions across all portal contexts.

5.1.1 5.1: Login Screen

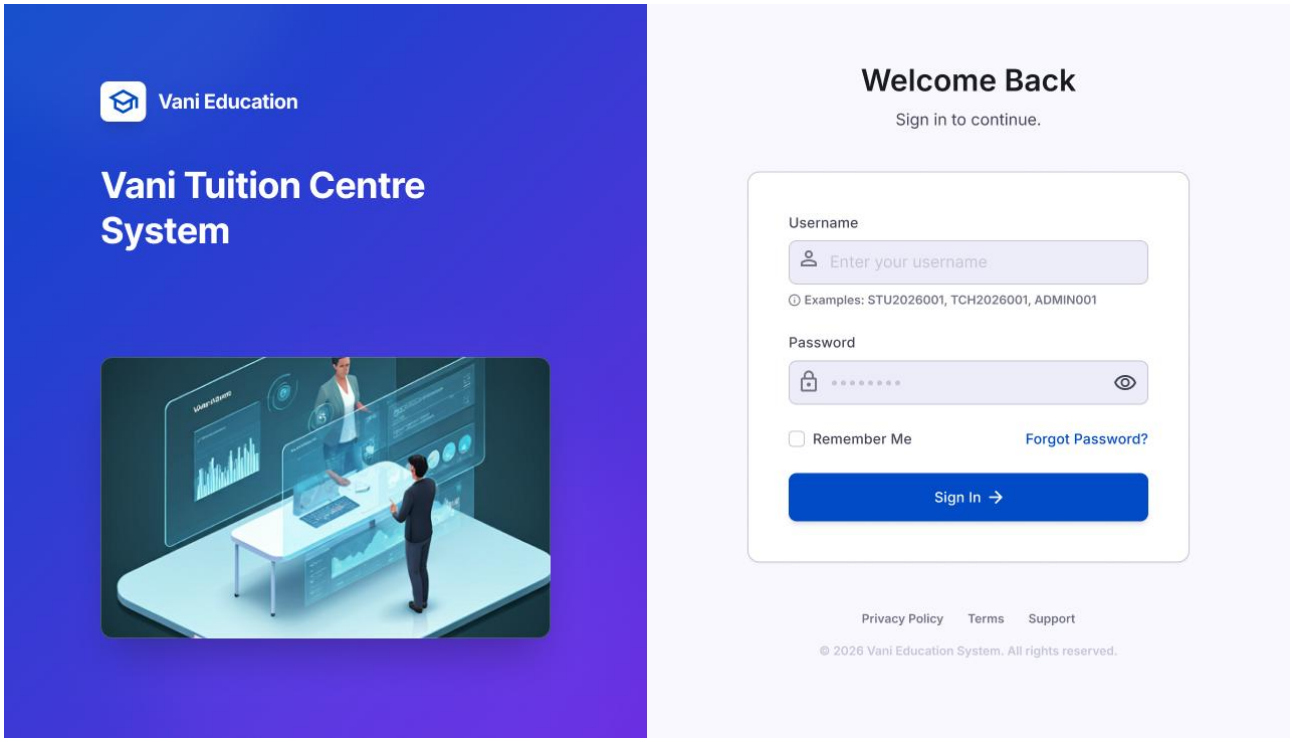

Figure 5.1 – Login Screen – Vani Tuition Centre Management System

Screen Name	Login Screen
Purpose	Single entry point for all user roles. Authenticates Admin, Teacher, and Student through a shared form.
Key Components	Logo and system name. Centered card with username field, password field, Login button, Forgot Password link.
Input Fields	Username (text, required). Password (text, masked, required).
User Actions	Submit credentials. Navigate to Forgot Password flow.
Validation Rules	Both fields required. Three failed attempts locks account for 15 minutes.
Navigation	Successful login → role-appropriate dashboard (Admin/Teacher/Student).

Table 5.1 – Screen Description – Login Screen

5.1.2 5.2: OTP Verification Screen

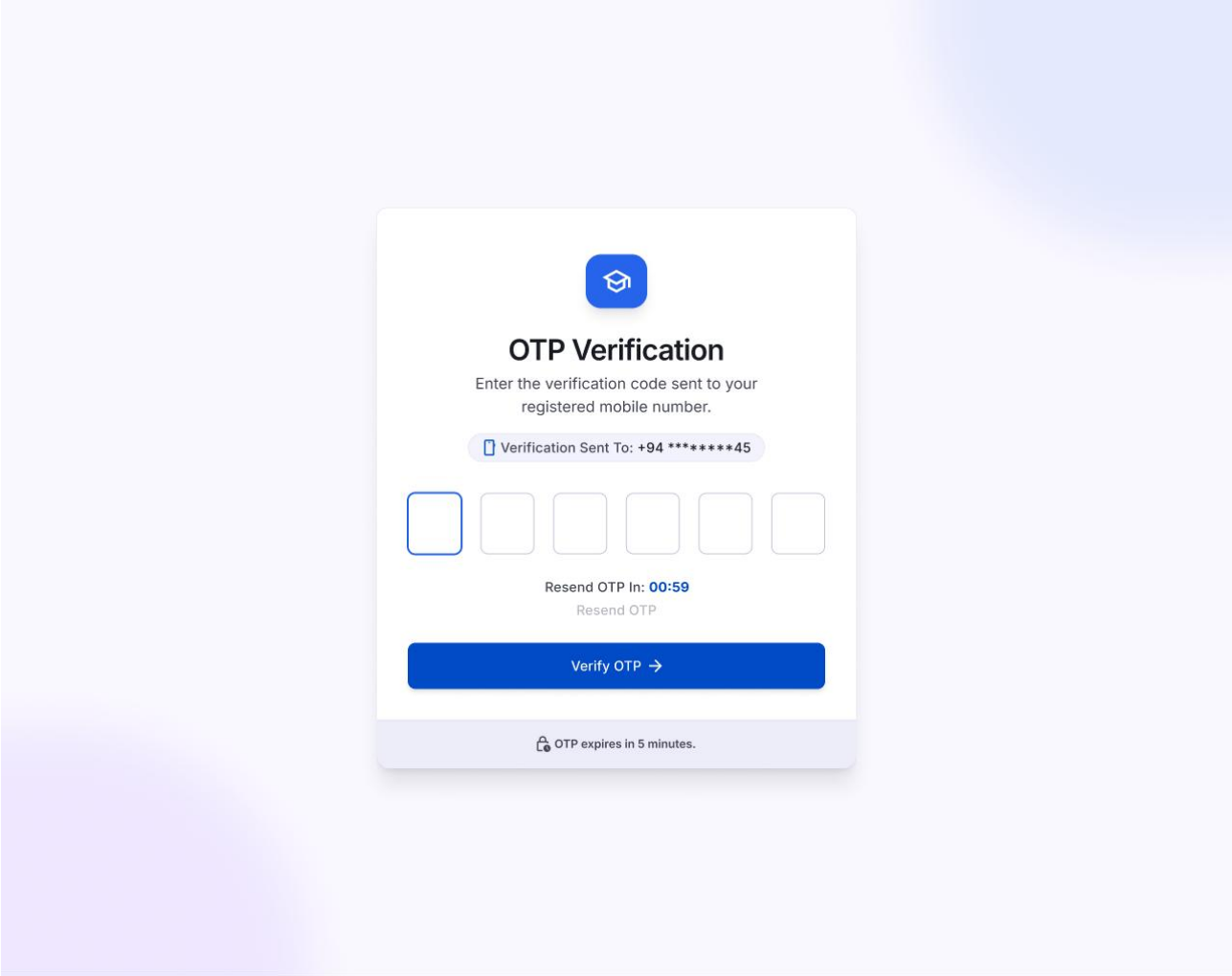

Figure 5.2 – OTP Verification Screen – Vani Tuition Centre Management System

Screen Name	OTP Verification Screen
Purpose	Supports password reset flow by confirming identity via one-time code sent to the registered email.
Key Components	Instruction text. 6-digit OTP input. Verify button. Resend OTP link with countdown timer.
Input Fields	OTP code (6-digit numeric, required).
User Actions	Submit OTP. Resend OTP (after timer expiry).
Validation Rules	Exactly 6 digits required. OTP expires after 10 minutes.
Navigation	Successful OTP → Reset Password Screen. Expired OTP → Login Screen.

Table 5.2 – Screen Description – OTP Verification Screen

5.1.3 5.3: Reset Password Screen

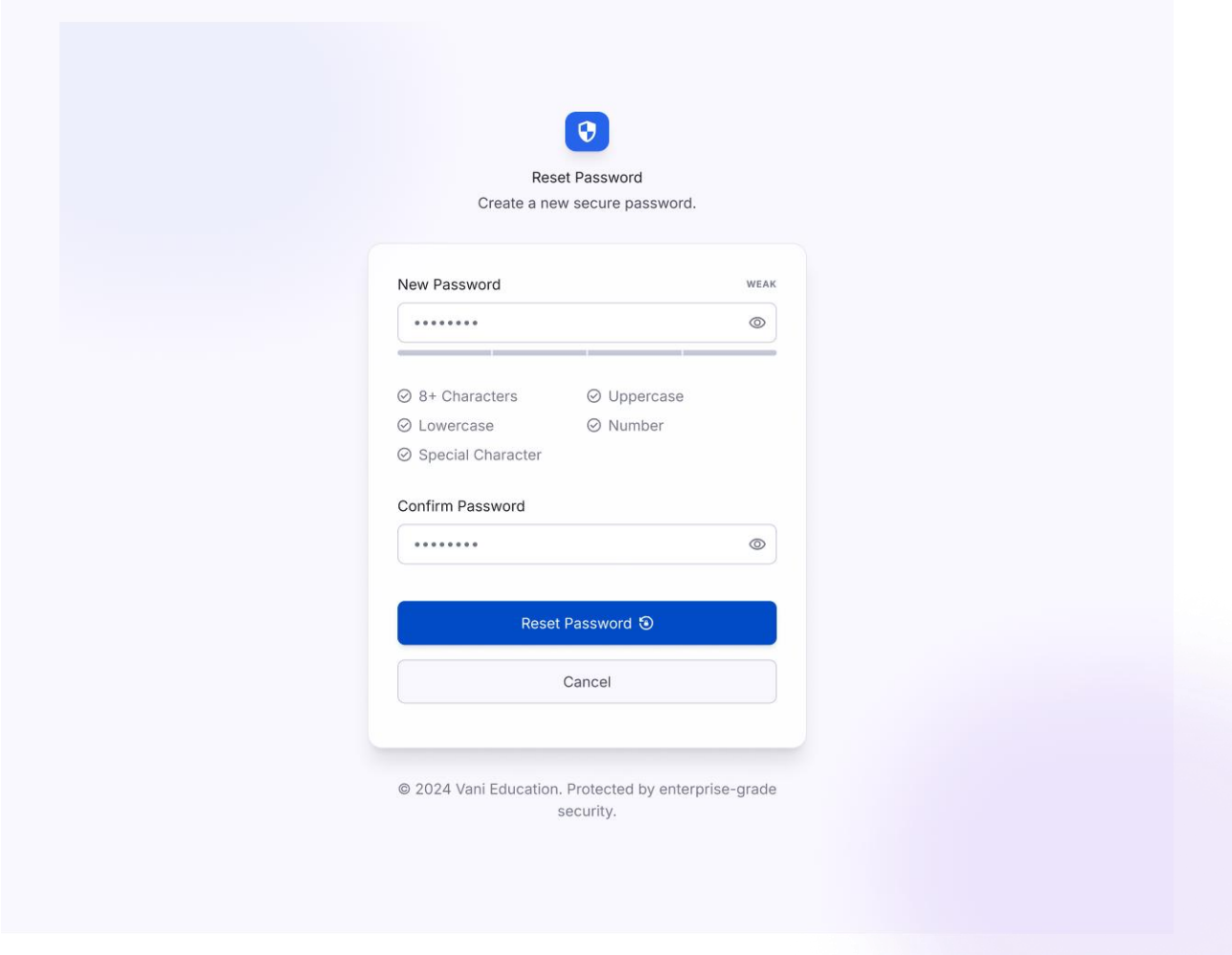

Figure 5.3 – Reset Password Screen – Vani Tuition Centre Management System

Screen Name	Reset Password Screen
Purpose	Allows a post-OTP-verified user to set a new account password.
Key Components	New Password field. Confirm Password field. Password strength indicator. Reset button.
Input Fields	New Password (masked, required). Confirm Password (masked, required).
User Actions	Submit new password.
Validation Rules	Minimum 8 characters. Must contain uppercase, lowercase, digit, and special character. Both fields must match.
Navigation	Successful reset → Login Screen with success notification.

Table 5.3 – Screen Description – Reset Password Screen

5.1.4 5.4: Admin Dashboard

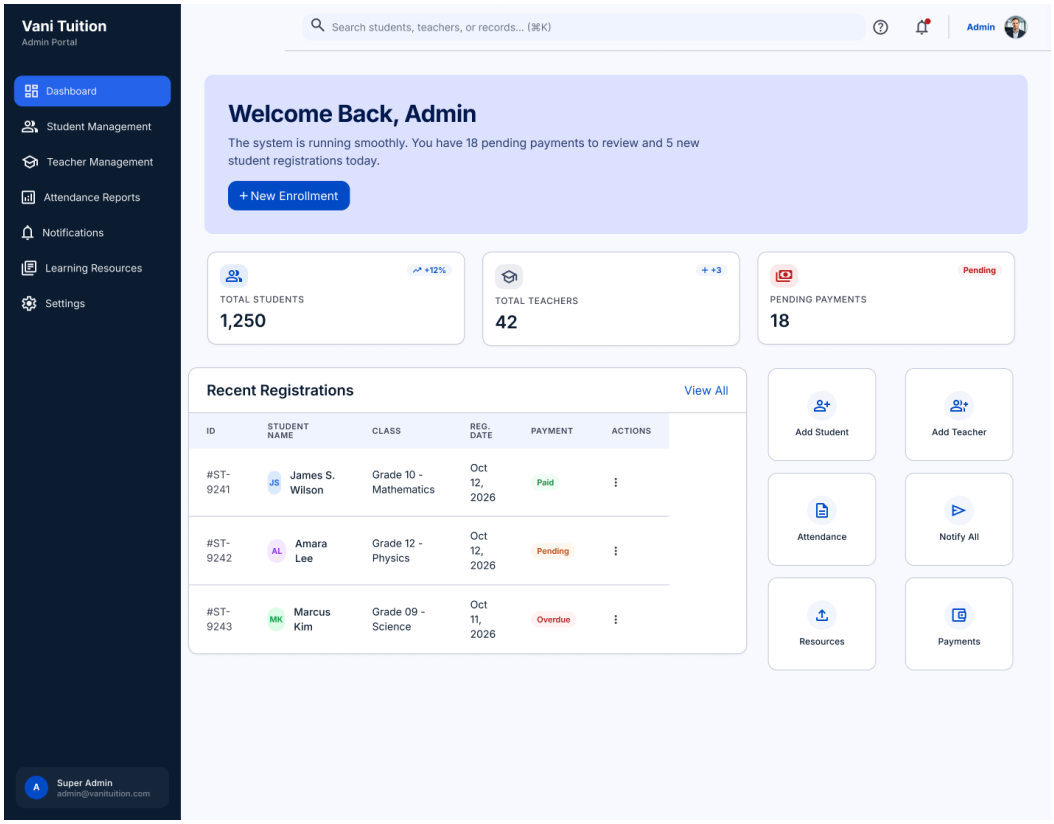

Figure 5.4 – Admin Dashboard – Vani Tuition Centre Management System

Screen Name	Admin Dashboard
Purpose	Central hub for the Admin Portal. Provides real-time operational metrics and navigation to all admin modules.
Key Components	Left sidebar with module links. Summary metric cards (students, teachers, attendance rate, pending payments). Recent activity feed. Quick action buttons.
Input Fields	None (read-only dashboard).
User Actions	Navigate to any admin module via sidebar. Drill into metrics.
Validation Rules	Not applicable.
Navigation	Sidebar → Student Management, Teacher Management, Attendance Reports, Payment Management, Notification Management, Learning Resources.

Table 5.4 – Screen Description – Admin Dashboard

5.1.5 5.5: Student Management Screen

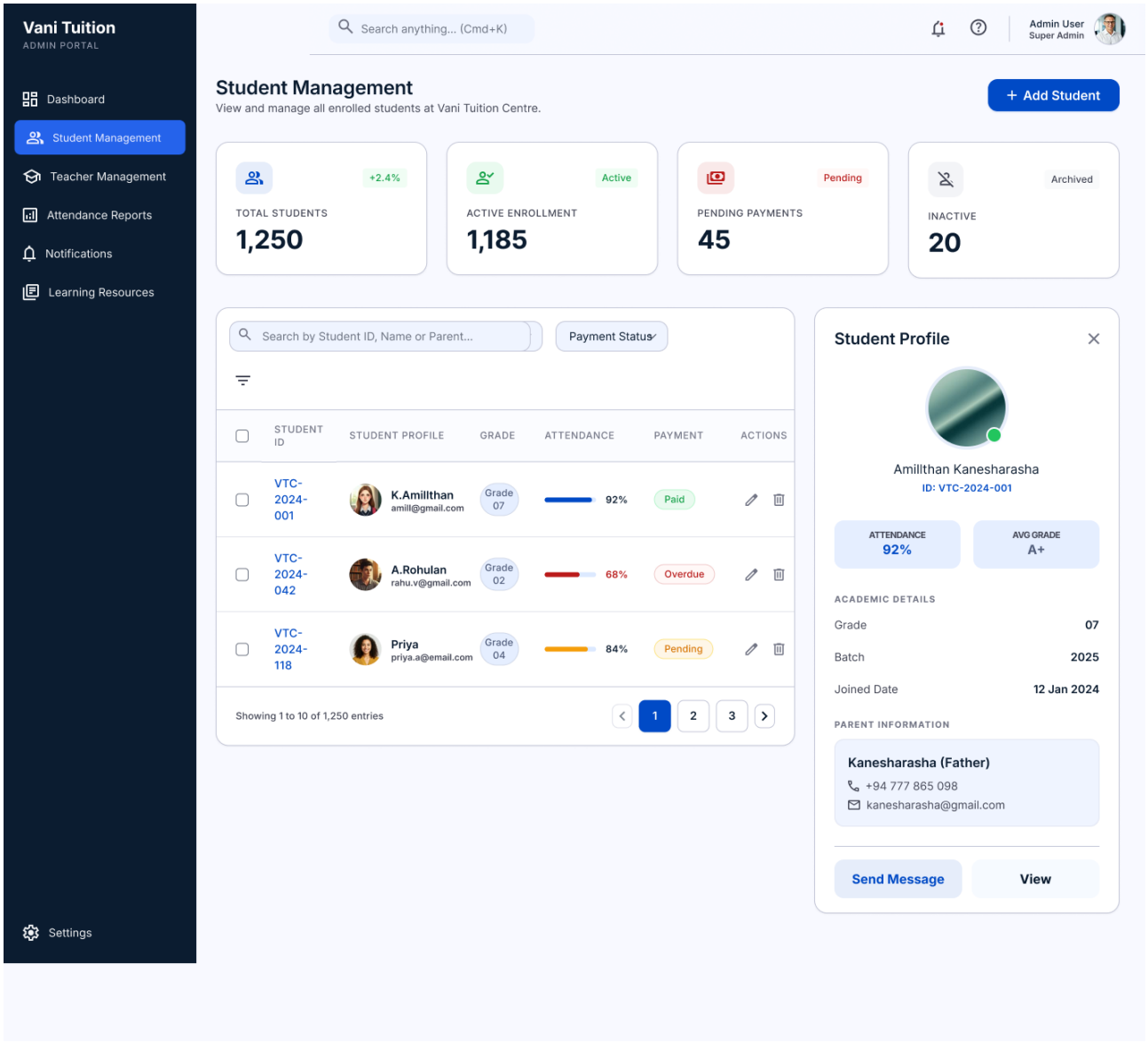

Figure 5.5 – Student Management Screen – Vani Tuition Centre Management System

Screen Name	Student Management Screen
Purpose	Tabular view of all enrolled students with search, filter, and full CRUD operation capabilities.
Key Components	Search bar. Add Student button. Paginated table (Student ID, Name, Parent Contact, Date, Status, Actions). Confirmation modal for deletions.
Input Fields	Search criteria. Student form fields on Add/Edit.
User Actions	Search, Add, Edit, Delete (confirmed), View student.
Validation Rules	Name: non-empty alphabetic. Phone: valid format. Email: valid email. DOB: past date.
Navigation	View → Student Details. Add → Registration Form.

Table 5.5 – Screen Description – Student Management Screen

5.1.6 5.6: Student Details Screen

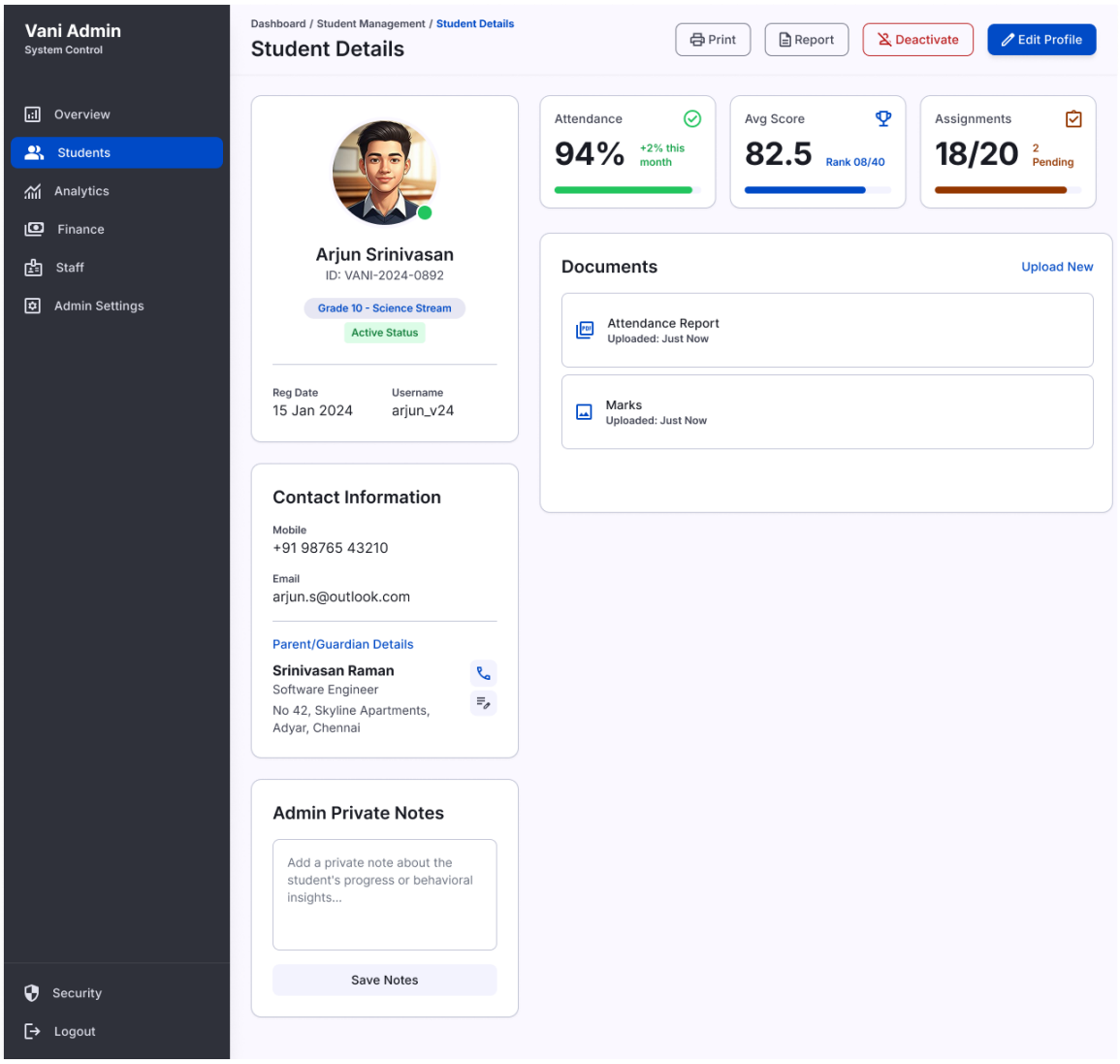

Figure 5.6 – Student Details Screen – Vani Tuition Centre Management System

Screen Name	Student Details Screen
Purpose	Displays comprehensive information for a single student including personal details, parent contacts, attendance summary, and payment history.
Key Components	Profile section. Parent contact section. Attendance summary table. Payment history table. Edit and Delete buttons.
Input Fields	None (read-only view).
User Actions	Edit student. Delete student. Navigate back.
Validation Rules	Not applicable in view mode.
Navigation	Edit → Student Edit Form. Delete → Student Management. Back → Student Management.

Table 5.6 – Screen Description – Student Details Screen

5.1.7 5.7: Teacher Management Screen

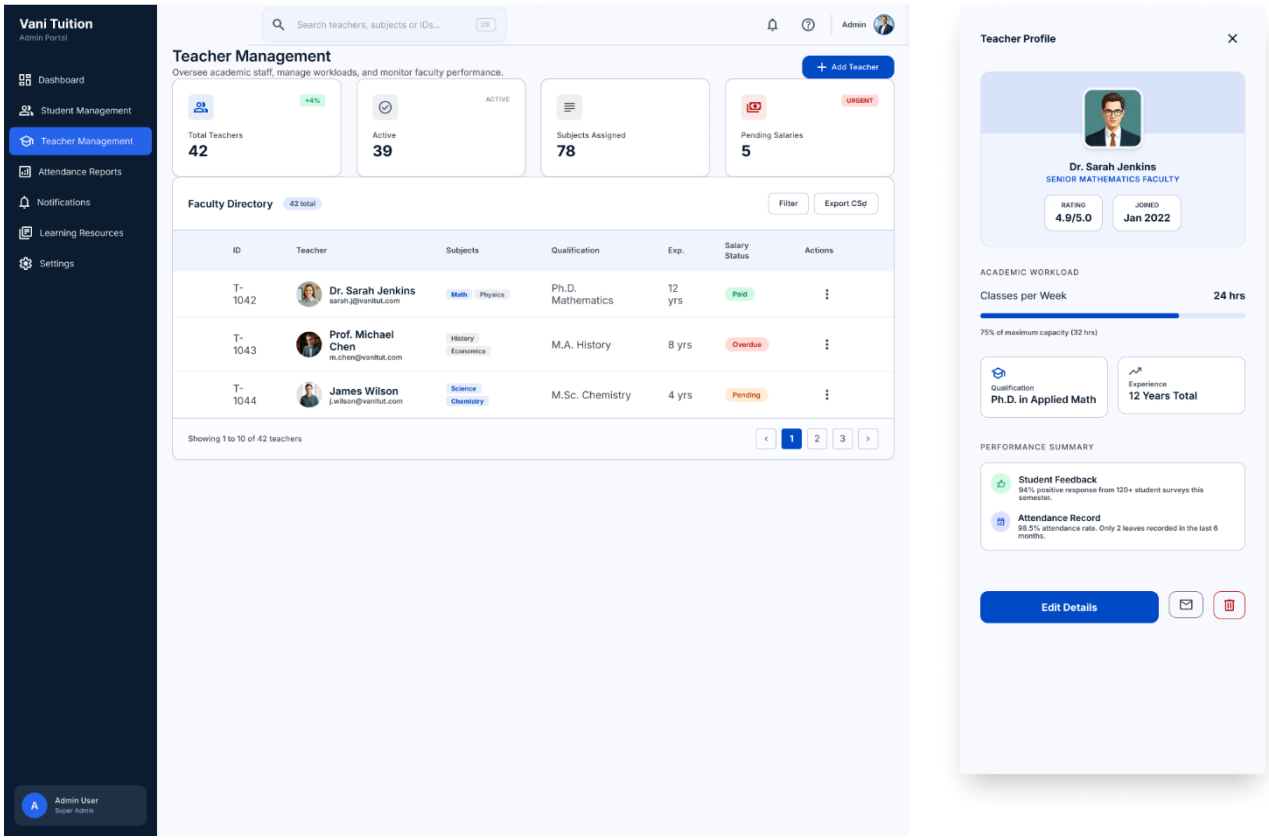

Figure 5.7 – Teacher Management Screen – Vani Tuition Centre Management System

Screen Name	Teacher Management Screen
Purpose	Comprehensive view and management of all teacher records, subject assignments, and salary payment tracking.
Key Components	Search bar. Add Teacher button. Paginated teacher table (ID, Name, Qualification, Subjects, Date, Status, Actions). Subject assignment modal.
Input Fields	Search criteria. Teacher form fields. Subject selection.
User Actions	Add, Edit, Delete (confirmed) teacher. Assign subjects. View payment history.
Validation Rules	Teacher name required. Qualification required. Salary must be positive decimal.
Navigation	View Details → Teacher Detail. Assign Subjects → Modal.

Table 5.7 – Screen Description – Teacher Management Screen

5.1.8 5.8: Attendance Reports & Analytics

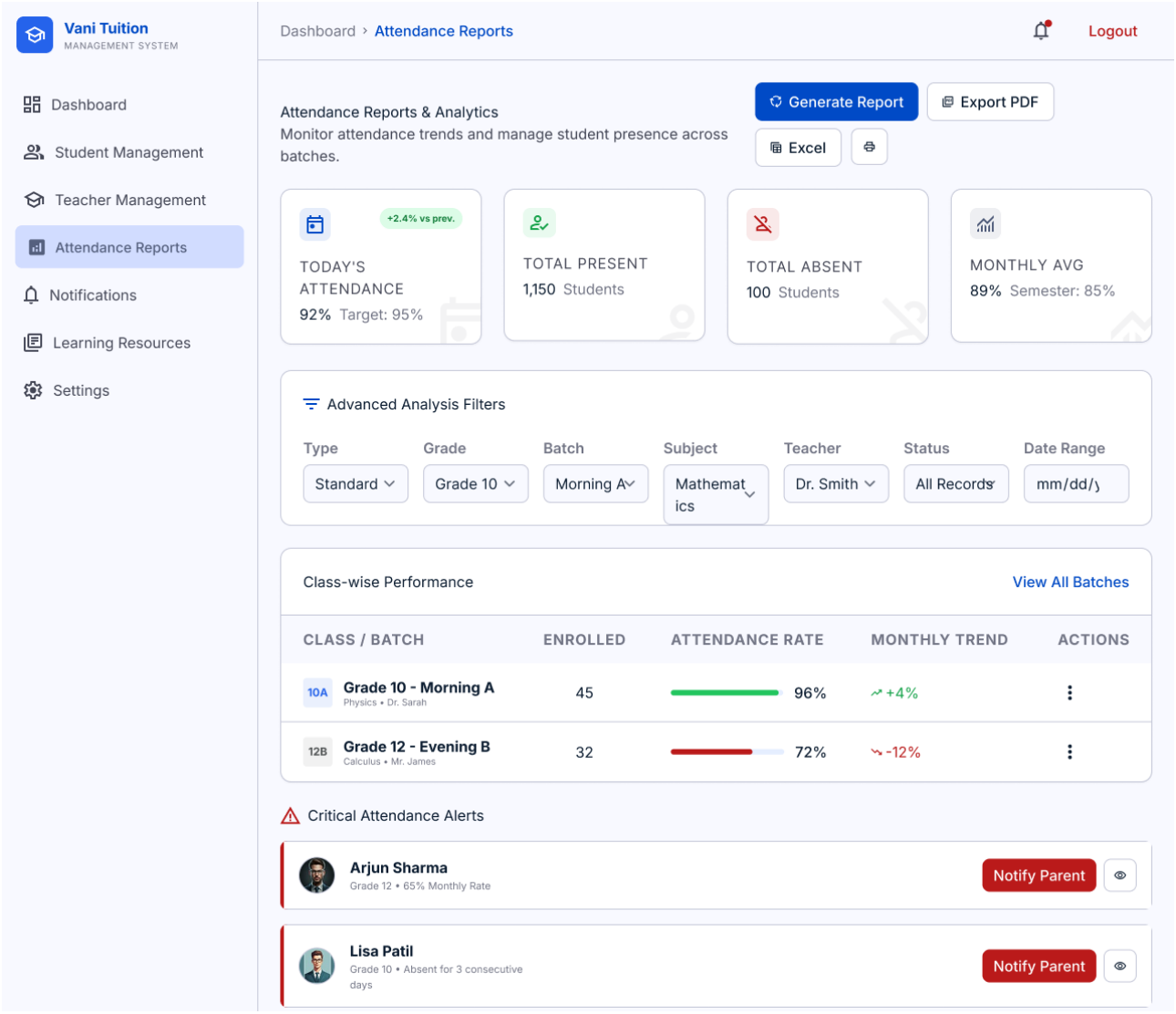

Figure 5.8 – Attendance Reports & Analytics – Vani Tuition Centre Management System

Screen Name	Attendance Reports & Analytics
Purpose	Enables Admin to generate, filter, and view attendance reports across the student population.
Key Components	Report criteria panel (student, date range, subject filter). Generate Report button. Summary table with attendance percentage metrics. Export option.
Input Fields	Student filter. Date range. Subject filter.
User Actions	Set criteria. Generate report. Export report.
Validation Rules	Start date must be before end date. At least one criterion required.
Navigation	Back → Admin Dashboard.

Table 5.8 – Screen Description – Attendance Reports & Analytics

5.1.9 5.9: My Attendance – Student Portal

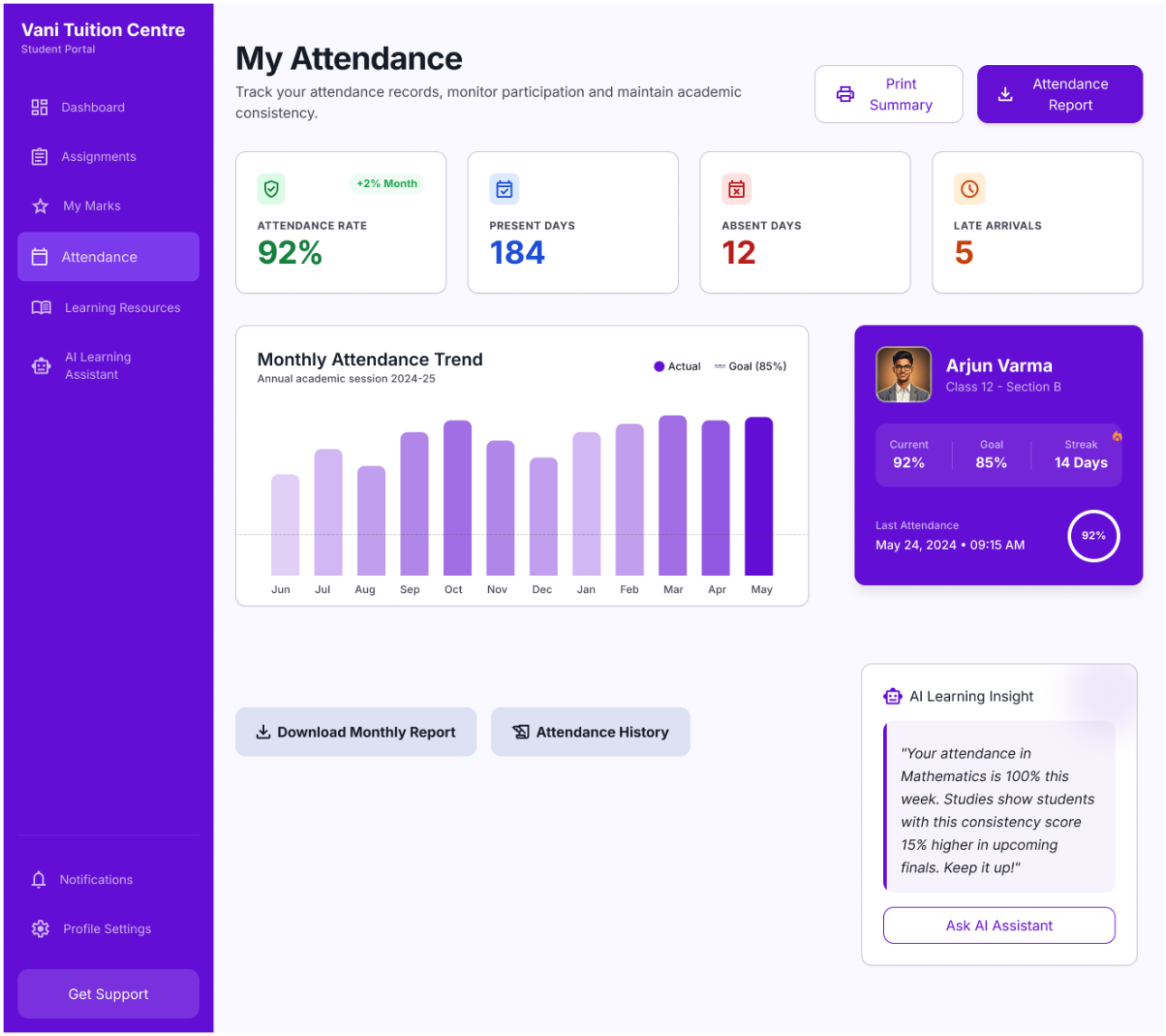

Figure 5.9 – My Attendance – Student Portal – Vani Tuition Centre Management System

Screen Name	My Attendance – Student Portal
Purpose	Student's personal attendance history view and QR code scanner for recording new attendance.
Key Components	Summary metrics (present count, absent count, percentage). QR Scanner button. Attendance history table. Monthly calendar heat map.
Input Fields	QR code (captured via camera scan).
User Actions	Scan QR code. View attendance history.
Validation Rules	QR must be valid and active. Each student can only scan once per session.
Navigation	Back → Student Dashboard.

Table 5.9 – Screen Description – My Attendance – Student Portal

5.1.10 5.10: Marks Management Screen

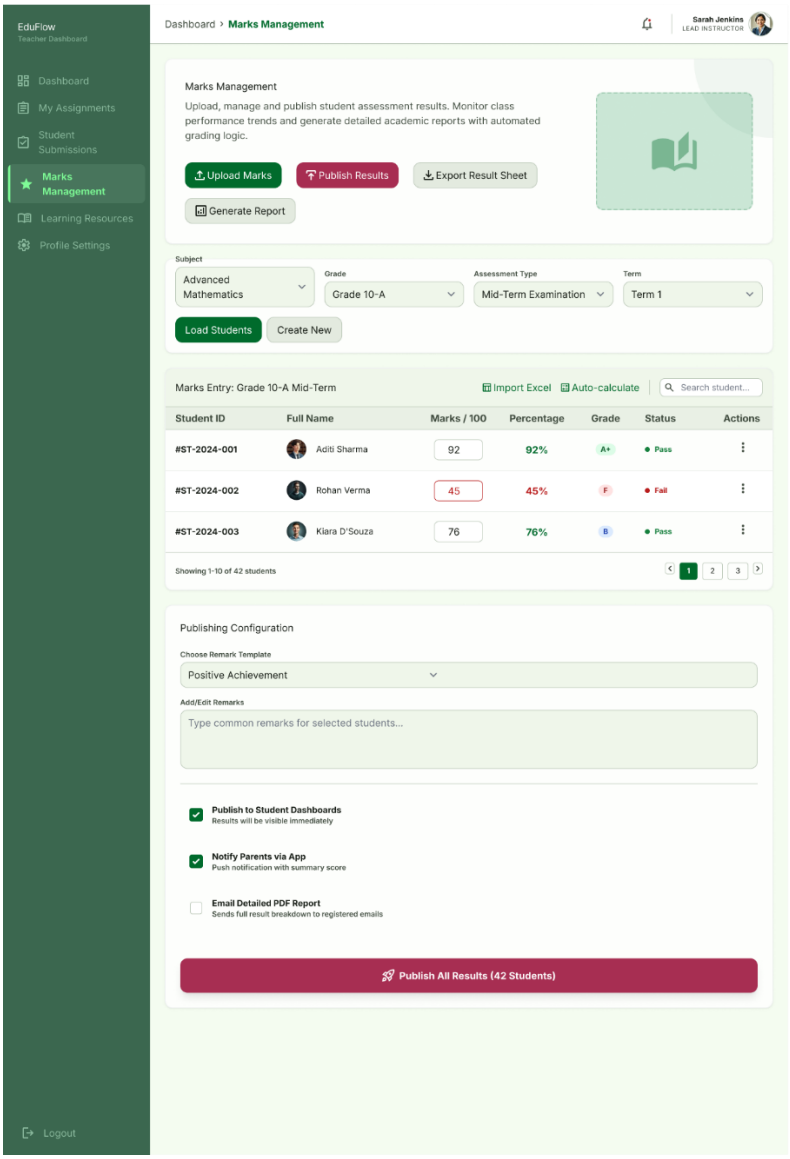

Figure 5.10 – Marks Management Screen – Vani Tuition Centre Management System

Screen Name	Marks Management Screen
Purpose	Teachers enter and upload marks. Students and parents view academic results with role-based visibility.
Key Components	Teacher view: Student selector, assessment name, score input, Upload button, marks table. Student view: Subject-filtered marks table.
Input Fields	Student (teacher). Assessment name. Score (numeric).
User Actions	Upload marks (teacher). View marks (student/parent).
Validation Rules	Score must be numeric within 0–100. Assessment name required.
Navigation	Back → Teacher Dashboard or Student Dashboard.

Table 5.10 – Screen Description – Marks Management Screen

5.1.11 5.11: My Assignments – Student Portal

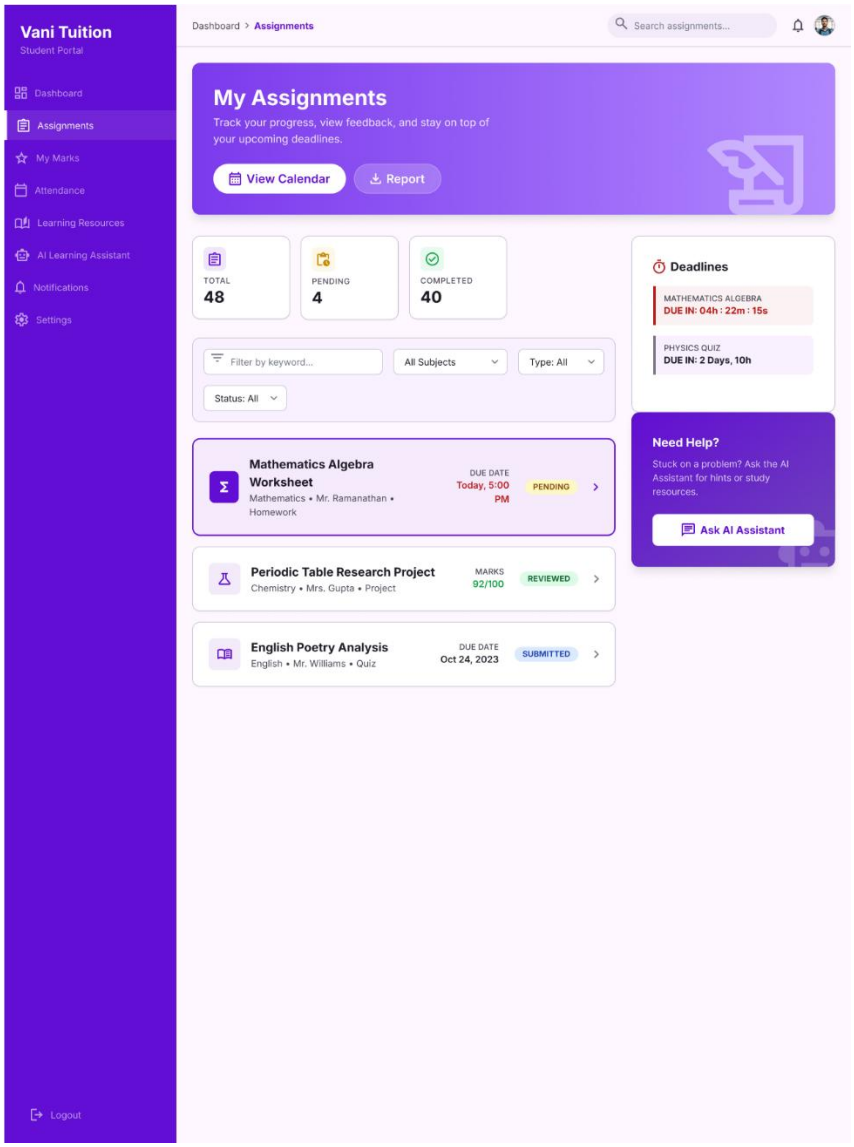

Figure 5.11 – My Assignments – Student Portal – Vani Tuition Centre Management System

Screen Name	My Assignments – Student Portal
Purpose	Displays published assignments to students with submission upload capability and status tracking.
Key Components	Assignment list (subject, title, deadline, status badge). Detail expansion area. File upload area. Submission status badge.
Input Fields	Submission file (document upload).
User Actions	View assignment details. Upload and submit assignment.
Validation Rules	File must not exceed size limit. Late submissions flagged but accepted.
Navigation	Back → Student Dashboard.

Table 5.11 – Screen Description – My Assignments – Student Portal

5.1.12 5.12: Teacher Assignment Management

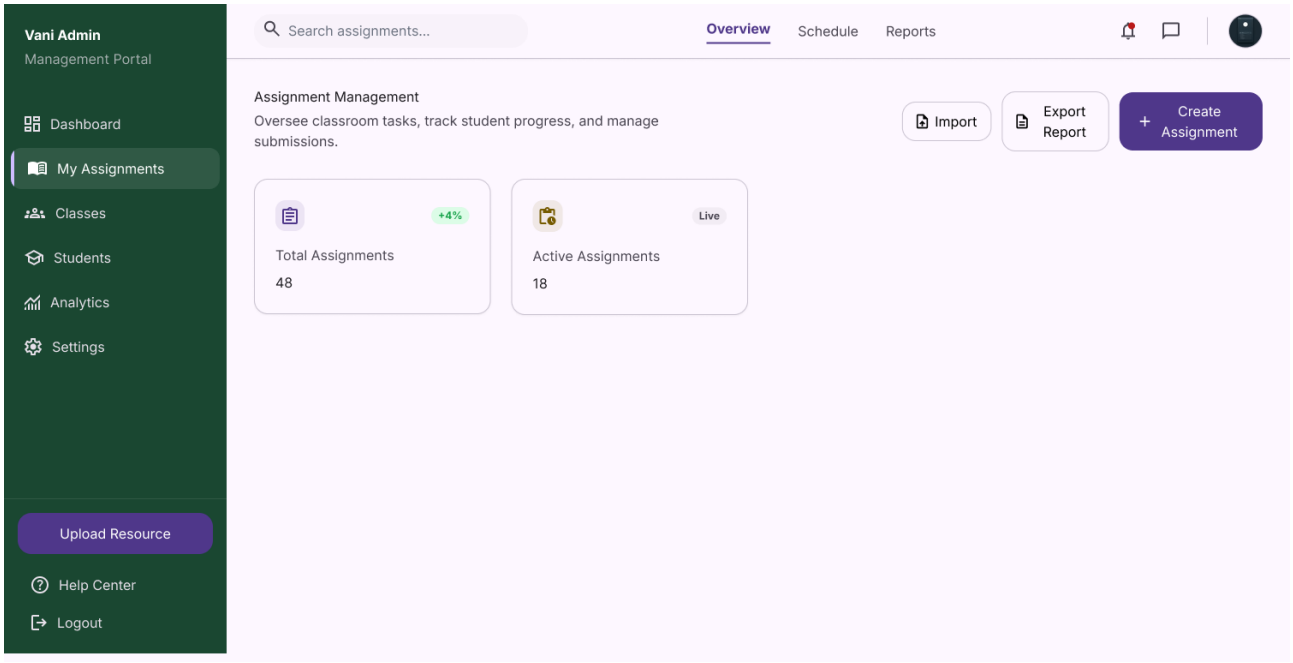

Figure 5.12 – Teacher Assignment Management – Vani Tuition Centre Management System

Screen Name	Teacher Assignment Management
Purpose	Teacher's management view of all created assignments with submission counts and review status.
Key Components	Assignment table (title, subject, deadline, submission count, status). Create New Assignment button. Review Submissions button per row.
Input Fields	None (list view).
User Actions	Create assignment. Review submissions. Edit or delete assignment.
Validation Rules	Not applicable on list view.
Navigation	Create → Create Assignment Form. Review → Submissions Review Screen.

Table 5.12 – Screen Description – Teacher Assignment Management

5.1.13 5.13: Create New Assignment Screen

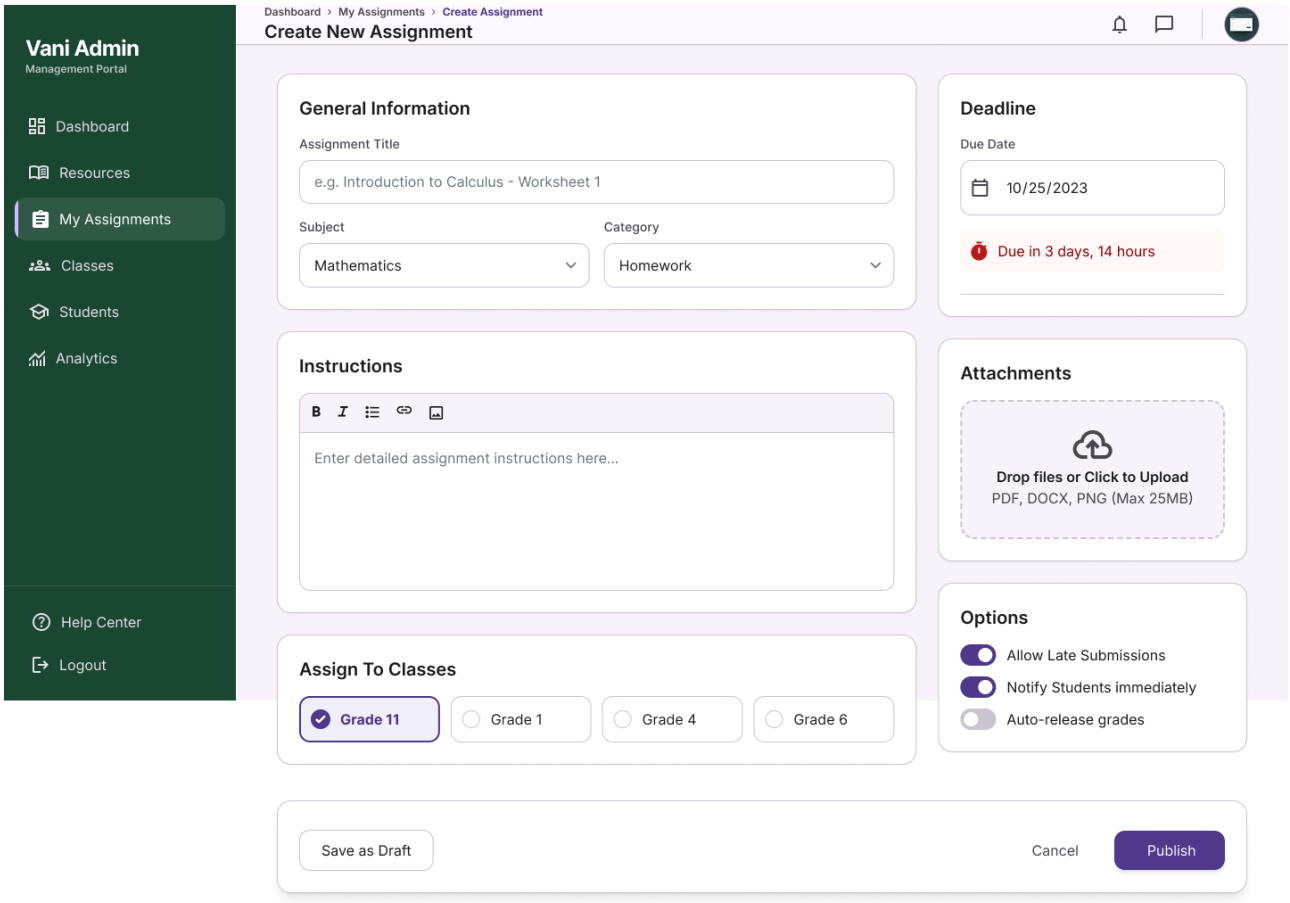

Figure 5.13 – Create New Assignment Screen – Vani Tuition Centre Management System

Screen Name	Create New Assignment Screen
Purpose	Form for teachers to create and publish a new assignment with file attachment and deadline.
Key Components	Title input. Subject selector. Description text area. File uploader. Deadline date-time picker. Publish and Save Draft buttons.
Input Fields	Title (required). Subject (required). Description. Attachment. Deadline (required).
User Actions	Save as draft. Publish immediately.
Validation Rules	Title and subject required. Deadline must be a future date-time.
Navigation	Cancel → Teacher Assignment Management. Publish → success confirmation.

Table 5.13 – Screen Description – Create New Assignment Screen

5.1.14 5.14: Notifications Management Screen

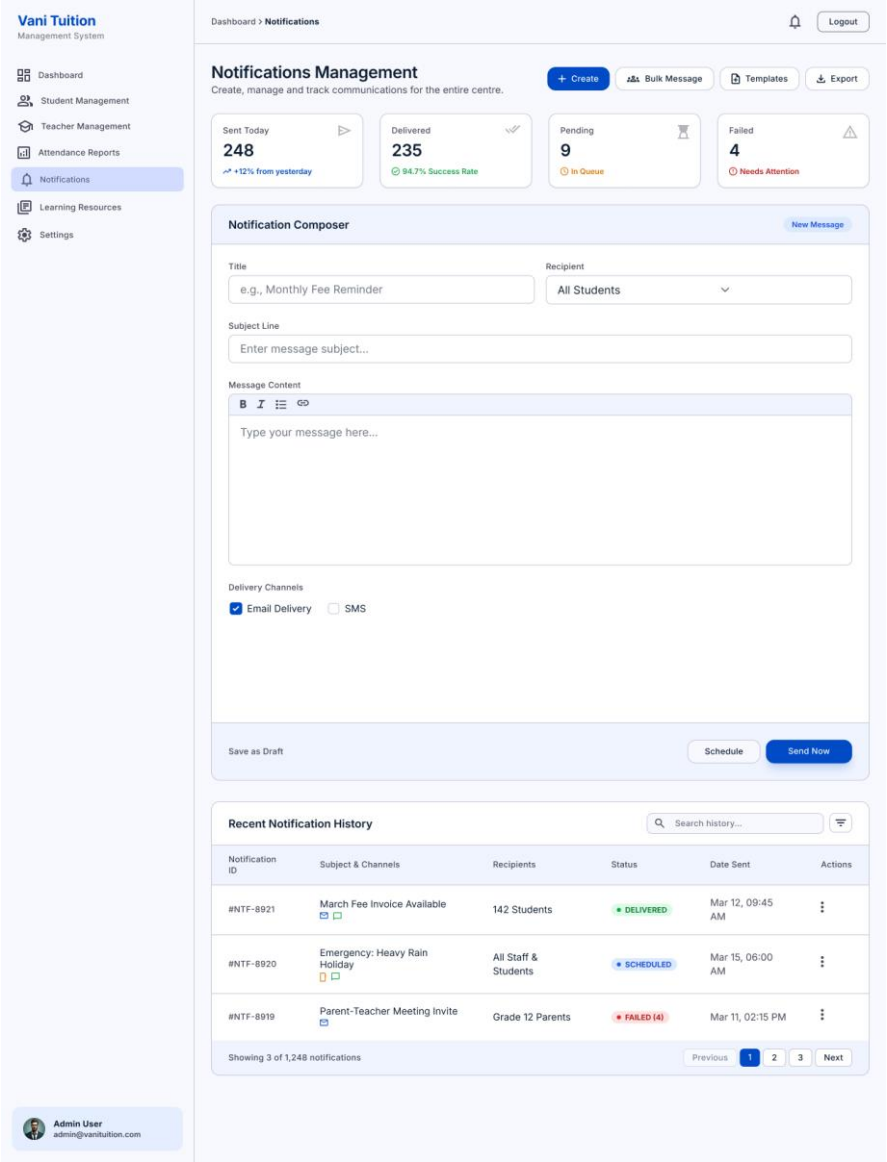

Figure 5.14 – Notifications Management Screen – Vani Tuition Centre Management System

Screen Name	Notifications Management Screen
Purpose	Admin creates, sends, and tracks targeted notifications to parents and students.
Key Components	Notification type selector. Recipient selector. Title and message inputs. Send button. Notification history table with delivery status.
Input Fields	Notification type. Recipients. Title. Message body.
User Actions	Compose and send notification. View history.
Validation Rules	Title and message required. At least one recipient required.
Navigation	Back → Admin Dashboard.

Table 5.14 – Screen Description – Notifications Management Screen

5.1.15 5.15: AI Learning Assistant Screen

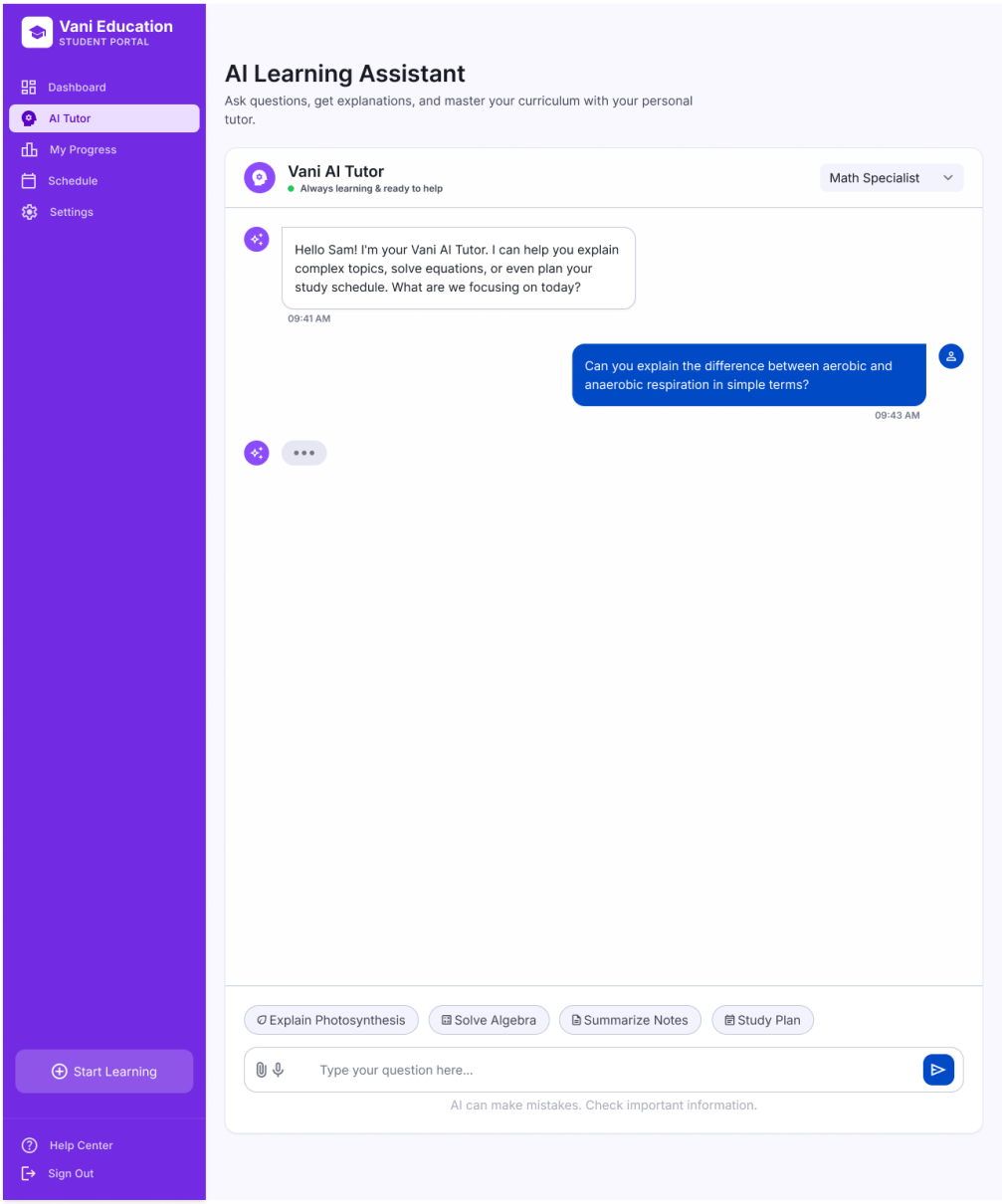

Figure 5.15 – AI Learning Assistant Screen – Vani Tuition Centre Management System

Screen Name	AI Learning Assistant Screen
Purpose	Interactive AI-powered chatbot for student academic support, searching the learning resource repository.
Key Components	Chat history area (student and AI message bubbles). Question input text area. Send button. Session clear button. Resource links within AI responses.
Input Fields	Student question (text).
User Actions	Submit question. Start new session. Clear history.
Validation Rules	Question must not be empty. Maximum length enforced.
Navigation	Back → Student Dashboard.

Table 5.15 – Screen Description – AI Learning Assistant Screen

5.1.16 5.16: Student Dashboard

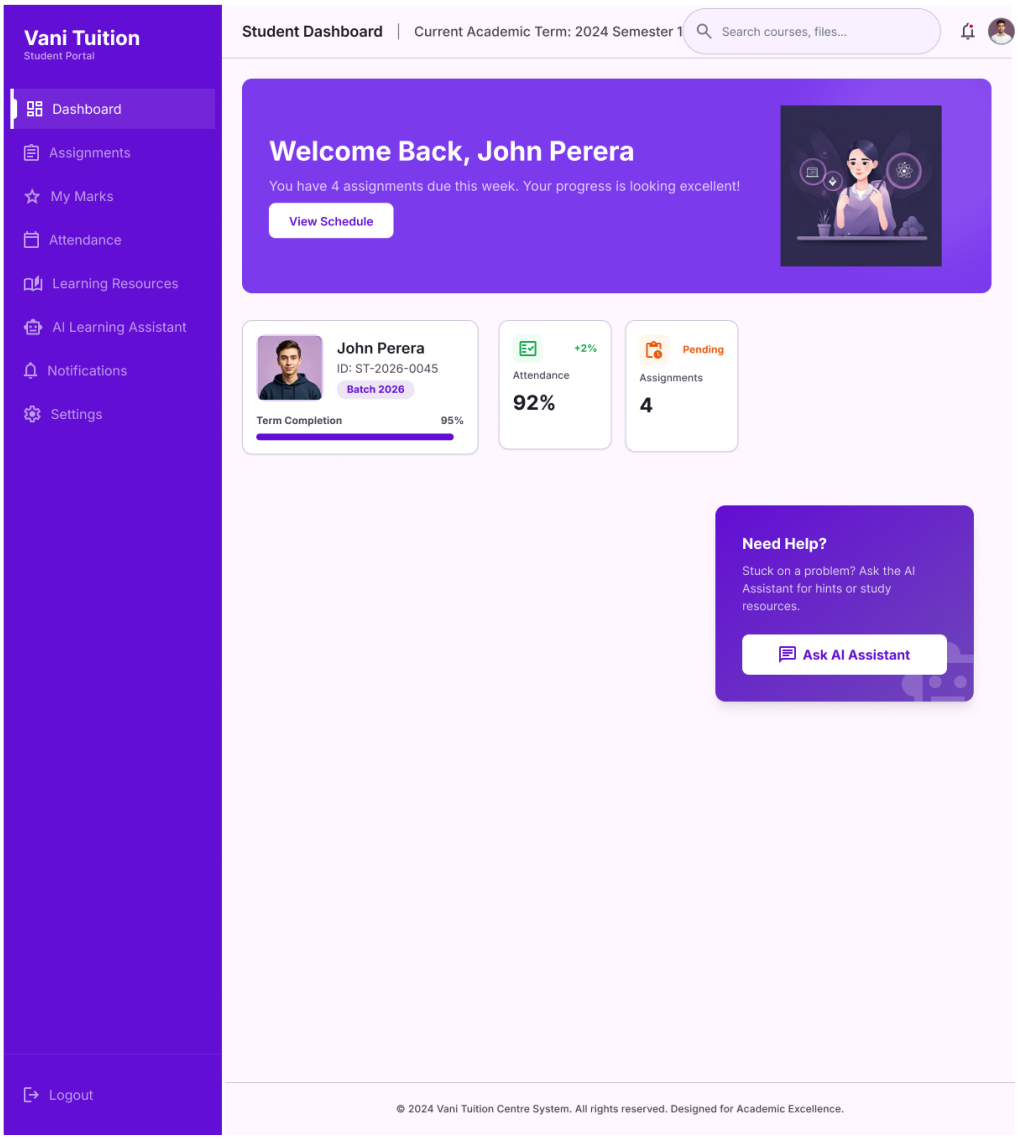

Figure 5.16 – Student Dashboard – Vani Tuition Centre Management System

Screen Name	Student Dashboard
Purpose	Central hub for the Student Portal. Personalised summary of academic status and quick access to all student features.
Key Components	Welcome banner. Metric cards (attendance %, assignment due count, unread notifications). Navigation links to all student modules. Recent activity feed.
Input Fields	None.
User Actions	Navigate to any student module.
Validation Rules	Not applicable.
Navigation	Cards → QR Attendance, Assignments, Marks, AI Chatbot, Timetable, Achievements.

Table 5.16 – Screen Description – Student Dashboard

5.1.17 5.17: Teacher Dashboard

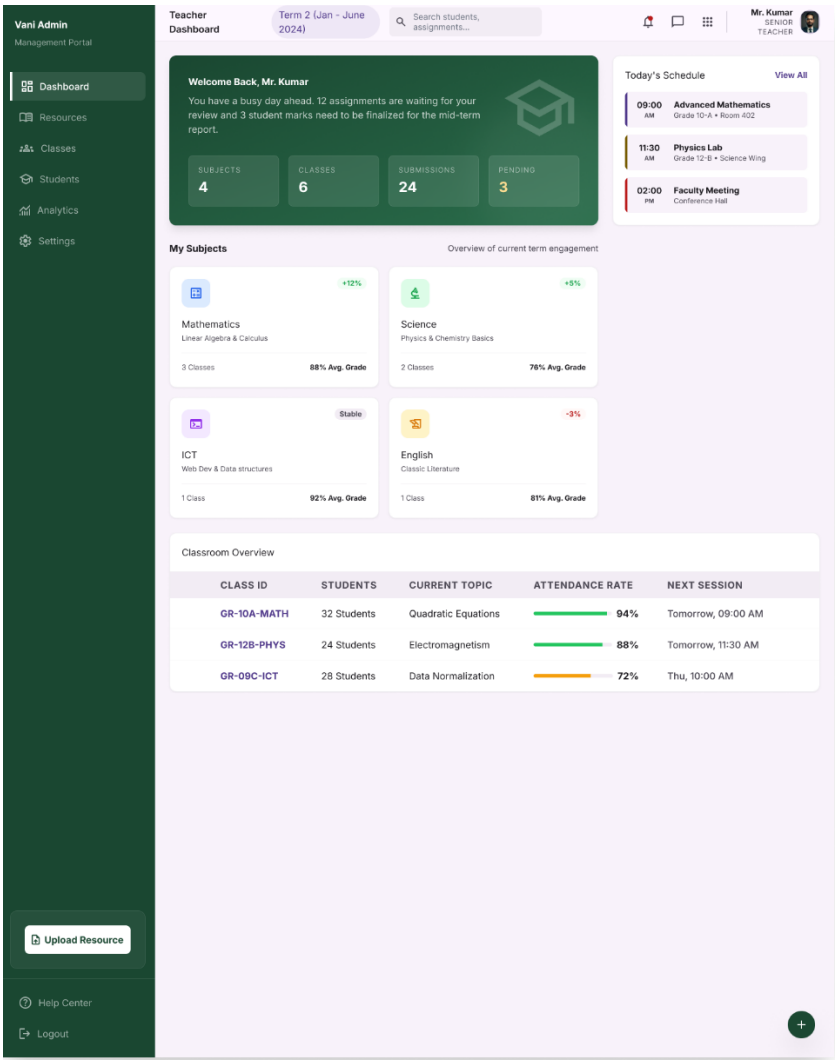

Figure 5.17 – Teacher Dashboard – Vani Tuition Centre Management System

Screen Name	Teacher Dashboard
Purpose	Teacher's overview of classes, assignment status, and marks upload activity.
Key Components	Metric cards (upcoming classes, pending reviews, marks to upload). Navigation to teacher modules. Recent submission activity feed.
Input Fields	None.
User Actions	Navigate to teacher modules.
Validation Rules	Not applicable.
Navigation	Navigation → Assignment Management, Marks Management, Review Submissions.

Table 5.17 – Screen Description – Teacher Dashboard

5.1.18 5.18: Home / Landing Page

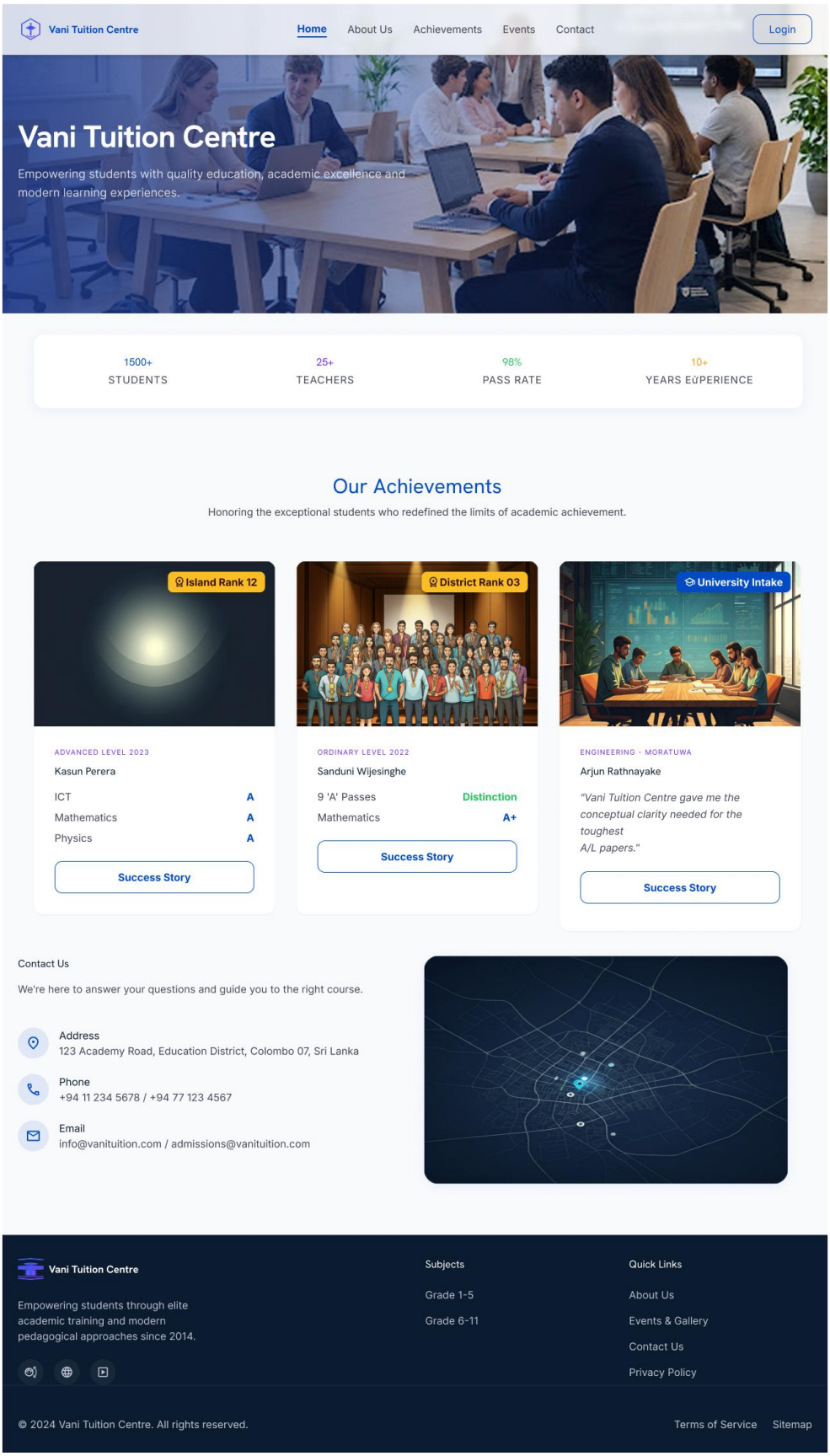

Figure 5.18 – Home / Landing Page – Vani Tuition Centre Management System

Screen Name	Home / Landing Page
Purpose	Public-facing landing page presenting the system's features and providing a login entry point.
Key Components	Navigation bar with Login button. Hero section with tagline. Feature highlights. Contact section.
Input Fields	None.
User Actions	Navigate to Login.
Validation Rules	Not applicable.
Navigation	Login button → Login Screen.

Table 5.18 – Screen Description – Home / Landing Page

5.1.19 5.19: Timetable Screen

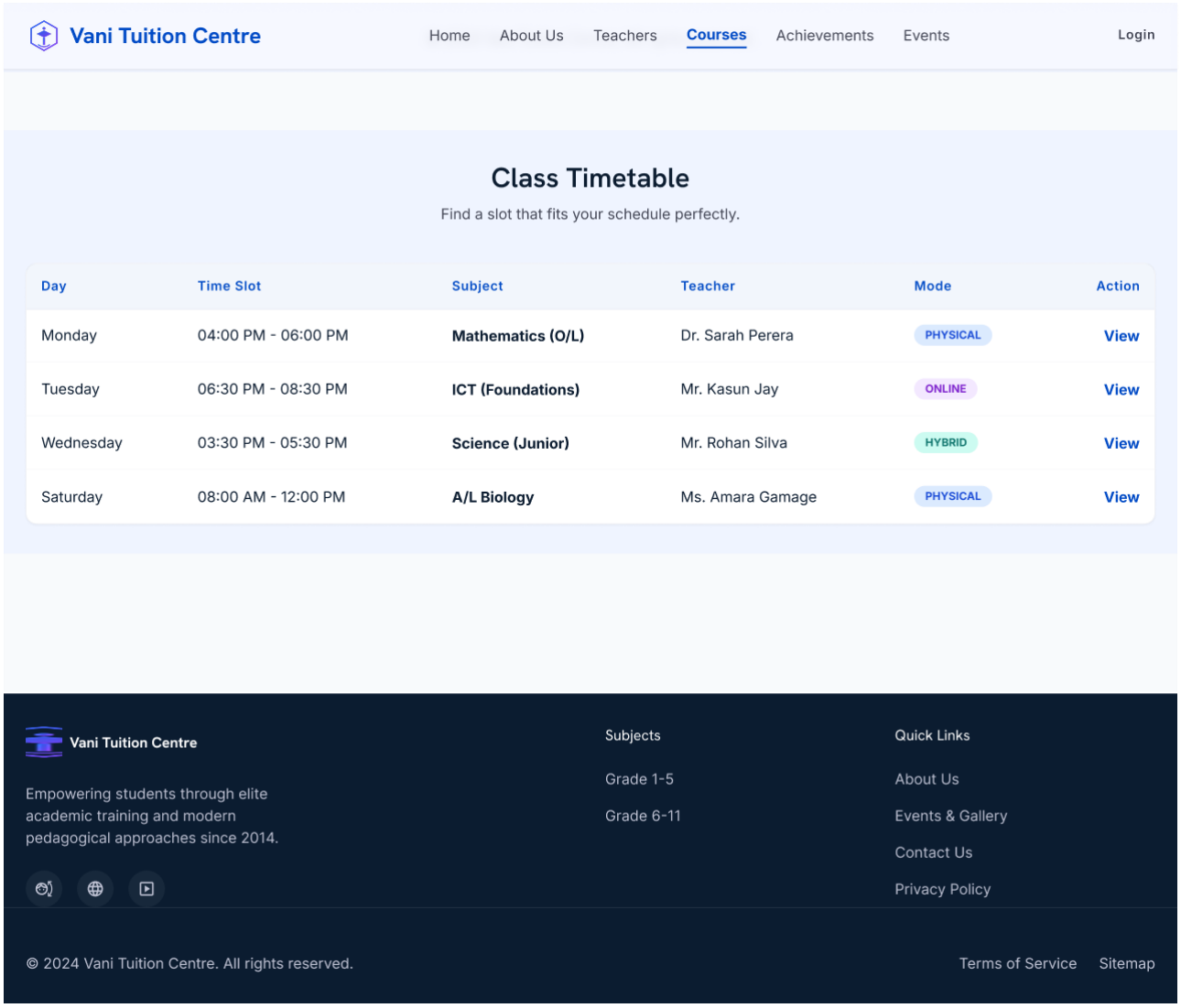

Figure 5.19 – Timetable Screen – Vani Tuition Centre Management System

Screen Name	Timetable Screen
Purpose	Displays the weekly class timetable for students and teachers to plan their schedule.
Key Components	Weekly calendar grid. Class session cards (subject, teacher, time). Day filter tabs.
Input Fields	None.
User Actions	View sessions. Filter by day.
Validation Rules	Not applicable.
Navigation	Back → respective dashboard.

Table 5.19 – Screen Description – Timetable Screen

5.1.20 5.20: Achievements Screen

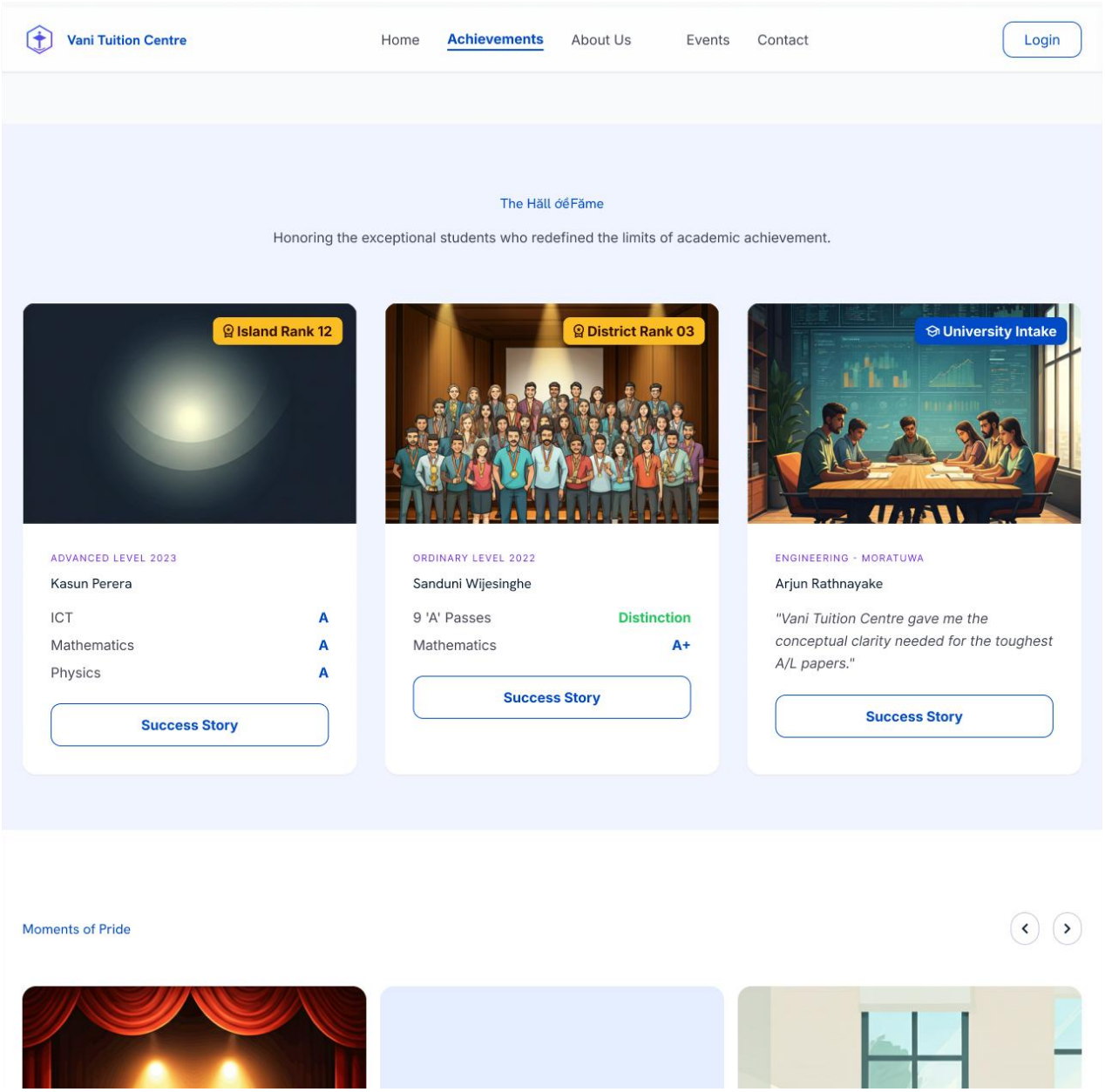

Figure 5.20 – Achievements Screen – Vani Tuition Centre Management System

Screen Name	Achievements Screen
Purpose	Displays academic achievements, badges, and milestones earned by students, gamifying the learning experience.
Key Components	Achievement badge gallery. Progress indicators. Achievement description tooltips.
Input Fields	None.
User Actions	View achievement details.
Validation Rules	Not applicable.
Navigation	Back → Student Dashboard.

Table 5.20 – Screen Description – Achievements Screen

5.1.21 5.21: Admin Settings Screen

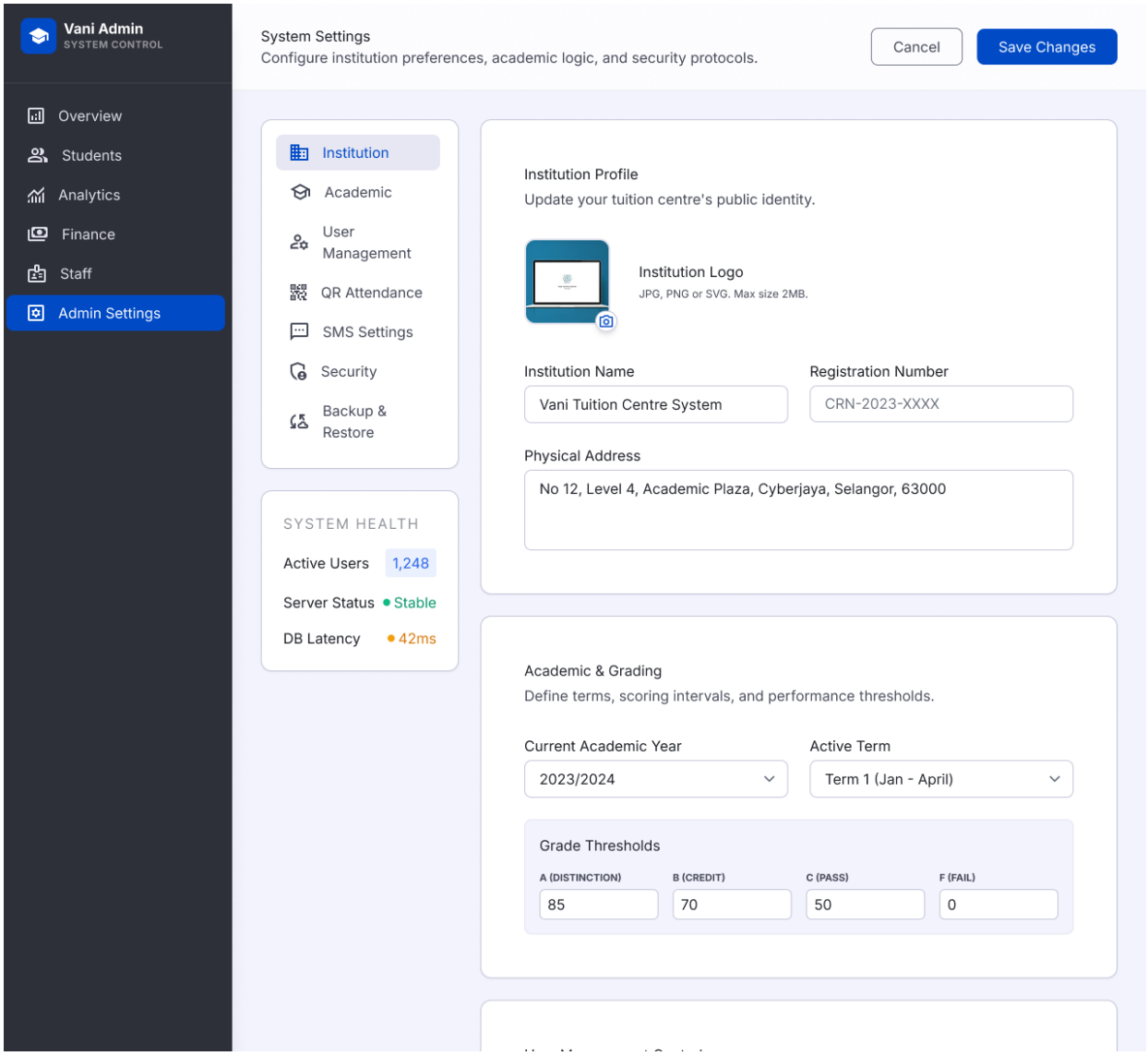

Figure 5.21 – Admin Settings Screen – Vani Tuition Centre Management System

Screen Name	Admin Settings Screen
Purpose	System configuration including account management, notification gateway settings, and QR session parameters.
Key Components	Account settings section. Notification gateway config. QR session expiry settings. System info section.
Input Fields	Configuration values per category.
User Actions	Update configuration. Save changes.
Validation Rules	Password change requires current password verification. Email format validation for gateway settings.
Navigation	Back → Admin Dashboard.

Table 5.21 – Screen Description – Admin Settings Screen

5.1.22 5.22: Upcoming Events Section

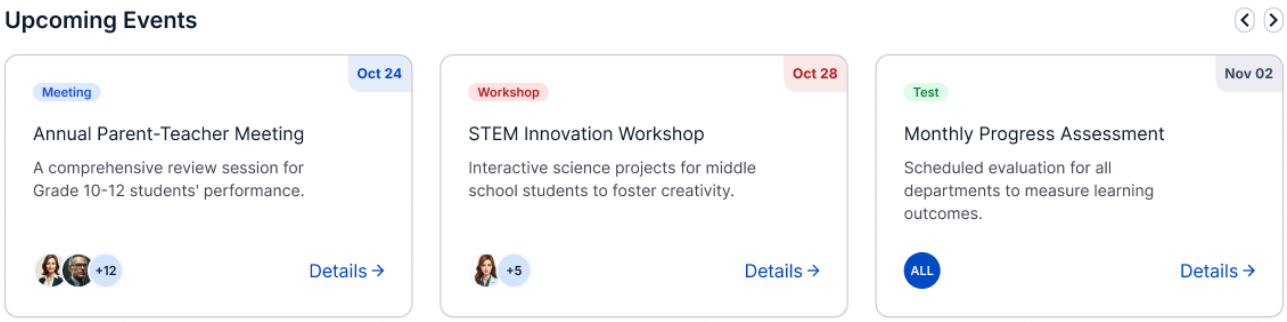

Figure 5.22 – Upcoming Events Section – Vani Tuition Centre Management System

Screen Name	Upcoming Events Section
Purpose	Dashboard widget displaying upcoming scheduled events (classes, exams, meetings) on portal dashboards.
Key Components	Event list with date, time, title, type icon. View All link.
Input Fields	None.
User Actions	View event details. Navigate to full events list.
Validation Rules	Not applicable.
Navigation	View All → Full Timetable or Events Page.

Table 5.22 – Screen Description – Upcoming Events Section

5.2 Navigation Flow Diagram

The Navigation Flow Diagram (Figure 5.23) illustrates the complete navigation paths available to each user role. The diagram is organised into three lanes — Admin Portal, Teacher Portal, and Student Portal — all sharing a common Login Page entry point.

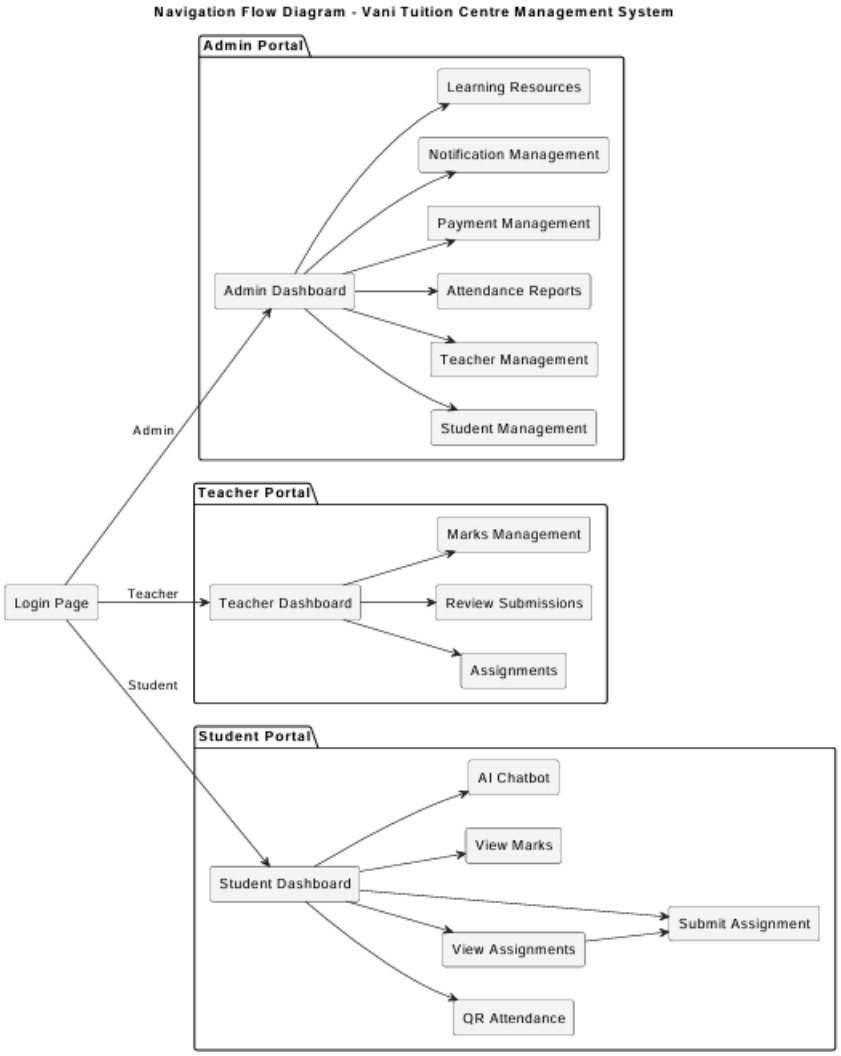

Figure 5.23 – Navigation Flow Diagram – Vani Tuition Centre Management System

Login Page: Single entry point for all three roles. After authentication, the system reads the JWT role claim and redirects to the appropriate portal dashboard.

Admin Portal: From the Admin Dashboard, the Admin navigates to Student Management (→ Student Details), Teacher Management, Attendance Reports, Payment Management, Notification Management, and Learning Resources. All modules are accessible directly from the sidebar.

Teacher Portal: From the Teacher Dashboard, the Teacher navigates to Assignment Management (→ Create Assignment, Review Submissions), Marks Management, and schedule views.

Student Portal: From the Student Dashboard, the Student navigates to QR Attendance, Assignments (→ submission upload), Marks, AI Chatbot, Timetable, and Achievements. The AI Chatbot is accessible as a persistent sidebar widget.

5.3 UX Design Decisions and Rationale

5.3.1 Usability

A persistent sidebar navigation pattern ensures module access is always one click away. Action buttons use standard iconography supplemented by text labels. Confirmation dialogs are presented before irreversible actions. Form fields display inline validation on blur rather than on submission, enabling real-time error correction.

5.3.2 Accessibility

All interactive elements meet minimum 44x44 pixel touch targets. Colour contrast meets WCAG 2.1 Level AA (4.5:1 for normal text, 3:1 for large text). Form fields are linked to descriptive label elements for screen reader compatibility. Error messages are announced via ARIA live regions.

5.3.3 Consistency

A shared design system governs all visual elements — typography, colour palette, spacing, and border radii — across all three portals. Primary actions use deep blue (#1F3A6E); destructive actions use red (#C0392B); neutral actions use grey. This colour coding trains users to recognise action severity without reading labels.

5.3.4 Mobile Responsiveness

The React SPA uses a mobile-first responsive layout strategy. On viewports narrower than 768px, the sidebar collapses to a hamburger menu. Tables convert to card-based layouts. The QR scanner feature is specifically optimised for mobile use, with enlarged tap targets and appropriately sized fonts.

5.3.5 Error Prevention

Assignment deadline fields default to 7 days in the future. Delete operations require two-step confirmation. Notification send buttons are disabled until at least one recipient is selected. QR attendance prevents double-scanning by marking a session as attended for the specific student on first successful scan.

5.3.6 User Experience Optimisation

Dashboards prioritise the most frequently needed information — the Admin sees student/teacher/payment metrics at a glance; the Teacher sees pending review counts; the Student sees attendance percentage and upcoming deadlines. Loading states use skeleton screens rather than spinners to reduce perceived wait time. Toast notifications provide non-disruptive success and error feedback.

6. INTERFACE DESIGN

6.1 External Interfaces

6.1.1 REST API – React Frontend to Spring Boot Backend

Attribute	Description
Purpose	Transfer authenticated, structured data between client and server for all system operations.
Protocol	HTTPS (HTTP over TLS 1.2/1.3). All traffic encrypted in transit.
Base URL Pattern	/api/v1/{resource}
Authentication	JWT Bearer Token in Authorization header. Validated on every request by Spring Security filter chain.
Input Format	JSON request body (POST/PUT). URL path parameters and query strings (GET/DELETE).
Output Format	JSON response body. HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.
Error Format	{ "error": "type", "message": "description", "timestamp": "ISO-8601" }

Table 6.1 – REST API Interface Specification

6.1.2 MongoDB Database Interface

Attribute	Description
Purpose	Persistent storage and retrieval of all system data.
Protocol	MongoDB Wire Protocol over TCP. Internal network only — not publicly exposed.
Connection	Connection pooling via Spring Data MongoDB auto-configuration. Credentials in encrypted environment variables.
Input Format	BSON documents. Queries through Spring Data repository methods or MongoTemplate.
Output Format	BSON documents mapped to Java entity objects via Spring Data @Document annotations.
Error Handling	MongoException caught by service layer and translated to standardised error responses.

Table 6.2 – MongoDB Interface Specification

6.1.3 SMS/Email Notification Gateway

Attribute	Description
Purpose	Delivery of attendance notifications, payment reminders, meeting notices, and announcements to parents.
Protocol	HTTPS REST API. API key authentication with gateway-issued credentials.
Input	POST with JSON payload: { "to": "+94XXXXXXXXX", "body": "Message text" } or email equivalent.
Output	Delivery acknowledgement with message_id and status. Async delivery status callbacks via webhook.
Failure Handling	Gateway error triggers FAILED status recording in notification_recipients. Error surfaced in Admin notification history.

Table 6.3 – Notification Gateway Interface Specification

6.1.4 AI Processing Engine

Attribute	Description
Purpose	Process student questions against learning resource context and generate educational responses.
Protocol	HTTPS REST API. API key authentication.
Input	POST with JSON: { "question": "Student query", "context": [resource excerpts array] }
Output	JSON: { "answer": "AI-generated response", "sources": [resource references] }
Failure Handling	Service unavailability returns a fallback message to the student: 'AI assistant temporarily unavailable, please try again.'

Table 6.4 – AI Processing Engine Interface Specification

6.2 Internal Module Interfaces

6.2.1 Authentication Module

Intercepts every API request (except /auth/* and /public/*). Extracts the JWT from the Authorization header, validates signature and expiry, and injects authenticated user identity and role into the Spring Security context. All subsequent modules consume this context without re-validating. Provides /api/v1/auth/login and /api/v1/auth/refresh endpoints.

6.2.2 Student Module

Exposes internal interfaces to the Attendance Module (student record lookup for attendance validation), Payment Module (payment-student association), and Notification Module (parent contact retrieval). All inter-module communication is in-process Java method calls through Spring-injected service dependencies.

6.2.3 Teacher Module

Provides interfaces to the Assignment Module (teacher-assignment association) and Marks Module (marks-teacher association). Consumes the Subject repository for subject assignment operations.

6.2.4 Attendance Module

Consumes the Student Module's student lookup interface. Produces attendance records consumed by the Report Module. Calls the Notification Module's sendNotification interface upon successful attendance recording, passing student ID and attendance timestamp.

6.2.5 Payment Module

Invoked by the Notification Module when generating payment reminders (to retrieve outstanding payment records). Provides read interfaces to the Admin Dashboard for summary metrics.

6.2.6 Notification Module

Provides a sendNotification(recipientId, notificationType, payload) interface consumed by the Attendance Module and Payment Module. The only module with a direct dependency on the external SMS/Email Gateway service.

6.2.7 Reporting Module

Aggregates data from the Attendance Module's repository to generate filtered reports. Pure read-only aggregation layer — no write dependencies. Reports are generated on demand from live database data without caching, ensuring accuracy.

7. SECURITY DESIGN

7.1 Authentication and Authorization Strategy

7.1.1 JWT Authentication

Authentication uses JSON Web Tokens (JWT) per RFC 7519. The /api/v1/auth/login endpoint validates credentials and issues a signed JWT containing: sub (user ID), role (ADMIN/TEACHER/STUDENT), iat (issued-at), and exp (24-hour expiry). The token is signed using HMAC-SHA256 with a server-held secret never transmitted to clients.

Clients store the JWT in browser memory (not localStorage or sessionStorage, which are XSS-vulnerable). Every API request includes the JWT in the Authorization Bearer header. The Spring Security JWT filter intercepts all requests, validates the token signature and expiry, and populates the security context. Requests with missing, malformed, or expired tokens receive a 401 Unauthorized response.

7.1.2 Password Hashing

Passwords are never stored in plaintext. All passwords are hashed using BCrypt with cost factor 12 at registration and password reset. BCrypt automatically incorporates a per-password salt (preventing rainbow table attacks) and is intentionally computationally expensive (preventing brute-force attacks). The stored 60-character hash includes the BCrypt version identifier, cost factor, and combined salt and hash.

7.1.3 Session Management

The system is fully stateless — no server-side sessions are maintained. The JWT is the complete session artifact. A refresh token flow is implemented: a long-lived (7-day) refresh token is issued alongside the short-lived (24-hour) access token and stored as an HttpOnly cookie. When the access token expires, the client uses the refresh token at /api/v1/auth/refresh to obtain a new access token without re-authentication.

7.1.4 Role-Based Access Control (RBAC)

RBAC is enforced at two levels. At the API level, Spring Security @PreAuthorize annotations restrict endpoints to specific roles — student management endpoints are annotated @PreAuthorize("hasRole('ADMIN')"). At the data level, service methods apply ownership checks — marks retrieval verifies the requesting student's ID matches the marks record's student_id. This dual enforcement prevents both horizontal privilege escalation (student accessing another student's data) and vertical privilege escalation (student accessing admin data).

Module	Admin	Teacher	Student
Student Management	Full CRUD	No Access	View Own Profile
Teacher Management	Full CRUD	View Own Profile	No Access
QR Attendance	View Reports	Generate QR	Scan QR
Marks Management	View All	Upload/Edit	View Own
Assignment Management	View All	Full CRUD	View/Submit
Notification Management	Full CRUD	No Access	View Received
Payment Management	Full CRUD	No Access	View Own Status
AI Learning Support	No Access	No Access	Full Access

Table 7.1 – Role-Based Access Control Matrix

7.2 Data Protection

7.2.1 Data at Rest

MongoDB access is restricted to the application server's IP through MongoDB's network access control. Credentials are stored in environment variables (never in source code). The MongoDB instance requires authentication for all connections. In production, MongoDB Encrypted Storage Engine (FIPS 140-2) would be enabled to encrypt data files on disk. Sensitive fields (passwords) are always stored as BCrypt hashes.

7.2.2 Data in Transit

All client-server communications are encrypted using TLS 1.2/1.3 (HTTPS). HTTP requests are redirected to HTTPS via server-level redirect rules. Application server to MongoDB communication occurs over an internal private network not accessible from the public internet. Communications with external services (notification gateway, AI engine) occur over HTTPS with certificate verification enabled.

7.2.3 Database Security

The application connects to MongoDB with a dedicated user having read/write access only to the VTCMS database — no administrative privileges. MongoDB port 27017 is not exposed to the public internet; it listens on the localhost interface and is accessible only from the application server through an internal network connection.

7.3 Input Validation Strategy

7.3.1 Validation Rules

All input validation is performed at the Spring Boot service layer using Java Bean Validation (JSR-303) annotations (@NotNull, @NotBlank, @Size, @Email, @Pattern). Server-side validation is enforced regardless of client-side validation state. Key rules include: username fields restricted to alphanumeric characters and underscores; email fields validated against RFC 5321 format; phone numbers validated against a country-specific pattern; score fields restricted to numeric values within 0–100; deadline fields validated as future date-times; file uploads restricted to approved MIME types and maximum size limits.

7.3.2 Sanitisation

All string inputs are sanitised before database storage. HTML tags are stripped using a whitelist-based HTML sanitiser to prevent stored XSS. MongoDB query parameters are constructed using Spring Data's parameterised query methods — user-supplied values are always treated as data, never as query operators, preventing NoSQL injection. File uploads are validated against their actual MIME type (not just the claimed extension) and renamed to a server-generated UUID upon storage, preventing path traversal attacks.

7.3.3 Error Handling

A global exception handler using Spring's @ControllerAdvice catches all unhandled exceptions. Stack traces are written to the server log (not exposed to clients) and translated into standardised error responses with an error code and safe, user-friendly message. Internal system details are never included in API error responses.

7.4 OWASP Top 10 Considerations

7.4.1 Injection

NoSQL injection is mitigated through Spring Data MongoDB's parameterised repository methods ensuring user-supplied values are never interpreted as MongoDB query operators. HTML injection and stored XSS are mitigated through server-side sanitisation stripping HTML from all string fields before persistence.

7.4.2 Broken Authentication

Addressed through JWT with 24-hour expiry, BCrypt password hashing with cost factor 12, account lockout after three consecutive failed logins, and HTTPS enforcement. The refresh token is stored as an HttpOnly cookie, inaccessible to JavaScript and resistant to XSS-based theft.

7.4.3 Sensitive Data Exposure

Mitigated by TLS for transit encryption, BCrypt for password storage, restricted database network access, and exclusion of sensitive fields (password hash, parent contact details for non-admin roles) from API response payloads.

7.4.4 Security Misconfiguration

Addressed through Spring Boot's production-mode configuration profile disabling debug endpoints, stack trace responses, and development-only features. CORS is configured with an explicit whitelist of allowed origins rather than a wildcard.

7.4.5 Broken Access Control

Addressed through dual-level RBAC — Spring Security annotations at the API level and ownership checks at the service level. IDOR is mitigated by verifying the requesting user's authenticated ID matches the resource's owner ID for all student-specific data retrieval operations.

7.4.6 Cross-Site Scripting (XSS)

Stored XSS prevented through server-side HTML sanitisation. Reflected XSS mitigated by React's default DOM escaping of all dynamic values (dangerouslySetInnerHTML is not used in the application). A Content-Security-Policy (CSP) HTTP header restricts the sources from which scripts can be loaded.

7.4.7 CSRF

Mitigated by using JWT Bearer token authentication in the Authorization header rather than cookie-based session tokens. Since the JWT is carried in a request header (not a cookie), cross-origin requests cannot be forged. The refresh token cookie is protected with the SameSite=Strict attribute preventing cross-origin transmission.

7.4.8 Logging and Monitoring

Comprehensive logging using SLF4J/Logback records: all authentication events (success, failure, token refresh, logout) with timestamps and client IPs; all API requests with authenticated user ID, HTTP method, endpoint, and response status; and all 403 Forbidden responses at WARN level. Logs are written to a persistent append-only log file. In production, logs would be forwarded to a centralised SIEM platform for real-time anomaly detection.

8. NON-FUNCTIONAL DESIGN DECISIONS

This section maps every non-functional requirement (NFR) from the Software Requirements Specification to a specific design decision made within this Software Design Specification. For each NFR, the table identifies the requirement description, the design decision taken in response, the implementation approach, and the justification explaining why the decision satisfies the requirement.

Requirement	Description	Design Decision	Implementation Approach	Justification
Performance – Response Time	API responses for standard read operations returned within 2 seconds under normal load.	React SPA with lazy code splitting. MongoDB indexed queries. Spring Boot HikariCP connection pooling.	React.lazy() reduces initial bundle size. MongoDB indexes on frequently queried fields (student_id, session_id) minimise query planning. Pre-warmed connection pool eliminates connection establishment latency.	Combined measures ensure data retrieval and rendering occur concurrently, keeping end-to-end response times within the 2-second target.
Scalability – Concurrent Users	System shall support at least 100 concurrent users without performance degradation.	Stateless JWT API enabling horizontal scaling. MongoDB replica set for read distribution.	Stateless REST API allows deployment of multiple application server instances behind a load balancer with no session affinity. MongoDB read preference set to secondary-preferred for report queries.	Stateless architecture enables linear scalability through additional server instances without architectural changes.
Availability – Uptime	System shall achieve 99% availability during operating hours.	Spring Boot Actuator health checks. MongoDB replica set with automatic failover.	Spring Boot Actuator exposes /health for load balancer health probes. Three-node MongoDB replica set provides automatic primary election on node failure.	Three replica nodes sustain one node failure without data unavailability. Health check endpoints enable automatic removal of unhealthy instances from the load balancer pool.
Security – Data Protection	All user data shall be protected from unauthorised access at rest and in transit.	JWT RBAC. BCrypt password hashing. HTTPS enforcement. MongoDB access control. Input sanitisation.	See Section 7 for full detail. Defence-in-depth: JWT enforces identity; RBAC enforces authorisation; BCrypt protects credentials; HTTPS encrypts transit; MongoDB access control restricts database-level access.	No single security layer failure results in complete system compromise.

Requirement	Description	Design Decision	Implementation Approach	Justification
Maintainability – Code Quality	System shall be structured to enable future enhancement with minimal rework.	Layered architecture. Repository pattern. REST API contract. Component-based frontend.	Spring Boot service/repository separation allows database migrations without affecting business logic. React component composition allows UI changes without affecting API integration.	Architectural boundaries reduce coupling, making each layer independently changeable without ripple effects.
Usability – Learnability	New users shall perform primary tasks without training documentation.	Consistent sidebar navigation. Inline validation feedback. Confirmation dialogs for destructive actions.	All three portals share identical navigation patterns. Forms provide real-time validation on blur. Dangerous operations require explicit two-step confirmation.	Consistent design patterns leverage existing user mental models from common web applications, reducing time to competency.
Portability – Browser Compatibility	System shall operate correctly on all modern browsers without browser-specific code.	React.js with Babel transpilation. CSS Flexbox and Grid layouts.	Babel transpiles modern JavaScript to ES5 for broad browser compatibility. Flexbox and Grid are supported by all browsers with global usage >1%.	Transpilation and progressive enhancement ensure consistent behaviour across Chrome, Firefox, Safari, and Edge on both desktop and mobile.
Reliability – Data Integrity	System shall never lose or corrupt data due to application-level errors.	MongoDB write concern majority. Application-level input validation. Global exception handling.	Write concern majority ensures write operations are acknowledged only after a majority of replica nodes confirm the write. Spring @Transactional annotations ensure atomic operations across multiple collections.	Write concern majority prevents data loss on primary node failure during a write. Atomic operations prevent partial writes leaving data in an inconsistent state.
Extensibility – New Features	New functional modules shall be addable without modifying existing modules.	Module-based service architecture. OpenAPI-documented REST endpoints.	Each domain is encapsulated in an independent Spring Boot service with its own repository. New services are added to the Spring context without modifying existing ones.	The Open/Closed principle applied at the service layer: existing services are closed for modification but open for extension through new service additions.

Requirement	Description	Design Decision	Implementation Approach	Justification
Accessibility – WCAG Compliance	System shall meet WCAG 2.1 Level AA accessibility guidelines.	Semantic HTML. ARIA labels. Full keyboard navigation. WCAG contrast ratio compliance.	React components use semantic HTML (button, nav, table, form). Interactive elements have descriptive ARIA labels. All UI navigable via keyboard alone. Colour palette validated against WCAG contrast requirements (4.5:1 normal text, 3:1 large text).	Semantic HTML and ARIA labels enable screen reader compatibility. Keyboard navigation ensures usability for users with motor impairments.

Table 8.1 – Non-Functional Requirements to Design Decision Mapping

The ten non-functional requirements addressed above span the full quality attribute spectrum. Performance and Scalability decisions ensure system responsiveness as the user base grows. Security decisions implement a comprehensive defence-in-depth strategy addressing credential theft, data interception, injection, access control bypass, and cross-site attack vectors. Maintainability and Extensibility decisions ensure future enhancement is feasible without full redesign. Usability and Accessibility decisions ensure the system is equitable and intuitive for all users. Portability decisions ensure browser-agnostic access. Reliability decisions ensure data durability and consistency across all operations.